

Play4Change – Aplicação de Aprendizagem Adaptativa para Educação Cívica

Radesh Ilesh Gamanbhai Govind

Orientadores: Nuno Datia
Michel Vorenhout

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

11 de Junho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

**Play4Change – Aplicação de Aprendizagem Adaptativa para Educação
Cívica**

A51620 Radesh Ilesh Gamanbhai Govind

Orientadores: Nuno Datia

Michel Vorenhout

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

11 de Junho de 2026

Resumo

O Play4Change é uma plataforma de aprendizagem adaptativa para o desenvolvimento de competências digitais segundo o referencial europeu DigComp 2.2, desenvolvida no âmbito do projeto U!REKA SHIFT (ISEL e Amsterdam University of Applied Sciences). Composta por um painel de administração web, uma aplicação móvel centrada num desafio diário e um servidor Spring Boot, a plataforma gera automaticamente conteúdo pedagógico por Inteligência Artificial (Mistral, via LangChain4j) a partir de documentos fornecidos por um formador, indexando-o em PostgreSQL/pgvector para deduplicação semântica. Quando um Learner erra, o sistema adapta o percurso de remediação e escala para tutoria conversacional com IA; a autenticação é *passwordless* e a aplicação foi validada por 50 ficheiros de teste automatizados.

Uma avaliação de usabilidade com doze utilizadores reais registou uma pontuação SUS média de 76,0, acima da média de referência da indústria (68,0), com a tutoria por IA como funcionalidade mais valorizada — um resultado que atesta a facilidade de uso, não o ganho de competências DigComp 2.2. Os resultados confirmam a viabilidade técnica de uma plataforma que gera e adapta autonomamente o seu conteúdo pedagógico.

Palavras-chave: Aprendizagem Adaptativa; Inteligência Artificial Generativa; *Retrieval-Augmented Generation*; Competências Digitais; DigComp 2.2.

Abstract

Play4Change is an adaptive learning platform for developing digital competences under the European DigComp 2.2 framework, developed as part of the U!REKA SHIFT project (ISEL and the Amsterdam University of Applied Sciences). Comprising a web administration panel, a mobile application centred on a daily challenge, and a Spring Boot server, the platform automatically generates pedagogical content via Artificial Intelligence (Mistral, via LangChain4j) from documents supplied by an instructor, indexing it in PostgreSQL/pgvector for semantic deduplication. When a Learner answers incorrectly, the system adapts the remediation path and escalates to conversational AI tutoring; authentication is passwordless, and the application was validated through 50 automated test files.

A usability evaluation with twelve real users recorded an average System Usability Scale (SUS) score of 76.0, above the industry reference average (68.0), with AI tutoring identified as the most valued feature — a result that attests to ease of use, not to DigComp 2.2 competence gains. The results confirm the technical feasibility of a platform that autonomously generates and adapts its own pedagogical content.

Keywords: Adaptive Learning; Generative Artificial Intelligence; Retrieval-Augmented Generation; Digital Competences; DigComp 2.2.

Utilização de Inteligência Artificial

No desenvolvimento deste projeto foram utilizadas as seguintes ferramentas de inteligência artificial generativa:

- **Google Gemini Flash e Gemini Pro** — utilizados principalmente na fase de investigação e enquadramento: pesquisa de conceitos, leitura e síntese de documentação técnica, e exploração de alternativas de design antes de tomar decisões de arquitetura. Foram também utilizados para *debater* e estudar tópicos novos — funcionando como interlocutor para clarificar dúvidas sobre padrões de domínio e comportamento de APIs externas.
- **Claude Sonnet (Claude Code e Claude Cowork)** — utilizados ao longo de todo o ciclo de desenvolvimento. O Claude Code (interface de linha de comandos) foi o principal auxiliar de programação: geração de esqueletos de código, sugestão e implementação de refactoros, diagnóstico e correção de bugs, e escrita de testes unitários. O Claude Cowork (interface de desktop) foi utilizado na fase de redacção do relatório: análise de saídas do Claude Code, geração de figuras de diagramas de sequência com matplotlib.
- **Prova de conceitos (*proof of concepts*)** — em várias ocasiões, as ferramentas de IA foram utilizadas para construir protótipos mínimos que permitiram validar a viabilidade técnica de uma abordagem antes de a integrar no projeto principal.

Em todos os casos, o código e o texto gerados foram revistos, iterados, adaptados e validados pelo autor.

Lista de Acrónimos e Siglas

Sigla	Significado
API	<i>Application Programming Interface</i>
CORS	<i>Cross-Origin Resource Sharing</i>
CSP	<i>Content Security Policy</i>
DAG	<i>Directed Acyclic Graph</i> (grafo acíclico dirigido)
DDD	<i>Domain-Driven Design</i>
DigComp	<i>Digital Competence Framework for Citizens</i>
DNS	<i>Domain Name System</i>
ECTS	<i>European Credit Transfer and Accumulation System</i>
HTTP(S)	<i>HyperText Transfer Protocol (Secure)</i>
IA	Inteligência Artificial
ITS	<i>Intelligent Tutoring System</i>
JPA	<i>Jakarta Persistence API</i>
JSON	<i>JavaScript Object Notation</i>
JVM	<i>Java Virtual Machine</i>
JWT	<i>JSON Web Token</i>
LLM	<i>Large Language Model</i>
MVI	<i>Model-View-Intent</i>
OCR	<i>Optical Character Recognition</i>
OWASP	<i>Open Worldwide Application Security Project</i>
RAG	<i>Retrieval-Augmented Generation</i>
RBAC	<i>Role-Based Access Control</i>
REST	<i>Representational State Transfer</i>
SPA	<i>Single Page Application</i>
SQL	<i>Structured Query Language</i>
SSE	<i>Server-Sent Events</i>
SSRF	<i>Server-Side Request Forgery</i>
SUS	<i>System Usability Scale</i>
TOCTOU	<i>Time-Of-Check to Time-Of-Use</i>
TTL	<i>Time To Live</i>
URL	<i>Uniform Resource Locator</i>
UUID	<i>Universally Unique Identifier</i>
XSS	<i>Cross-Site Scripting</i>

Índice

1	Introdução	1
1.1	Contextualização	1
1.2	Motivação	1
1.3	Objetivos	2
1.4	Âmbito, Limitações e Organização do Documento	3
2	Estado da Arte	5
2.1	Aprendizagem Adaptativa e Sistemas de Tutoria Inteligente	5
2.2	Inteligência Artificial na Educação	5
2.2.1	O Problema dos 2 Sigma e a Escalabilidade da Tutoria	6
2.2.2	Feedback Formativo e Aprendizagem Visível	6
2.2.3	Teoria da Carga Cognitiva	6
2.3	Aprendizagem Formal, Informal e Móvel	6
2.4	Retrieval-Augmented Generation	7
2.4.1	Arquitetura RAG	7
2.4.2	RAG em Contextos Educativos	7
2.5	Autenticação <i>Passwordless</i>	7
2.6	Síntese Comparativa	8
3	Análise e Requisitos	9
3.1	Identificação dos Utilizadores	9
3.1.1	Administrador	9
3.1.2	Learner	9
3.2	Requisitos Funcionais	10
3.3	Requisitos Não Funcionais	11
3.4	Casos de Uso Principais	12
3.4.1	CU01 — Submeter Conteúdo e Gerar Desafios	12
3.4.2	CU02 — Responder ao Desafio Diário	12
3.4.3	CU03 — Remediação Adaptativa (Struggle Path)	13

4	Arquitetura e Fundações	15
4.1	Arquitetura Geral do Sistema	15
4.1.1	Visão Macro e Estilo Arquitetural	15
4.1.2	Decomposição em Módulos	16
4.1.3	Camadas do Sistema	16
4.1.4	Domínio: Contextos Delimitados	17
4.2	Backend — Spring Server e Pipeline	18
4.2.1	Organização do Projeto	18
4.2.2	Pipeline de Processamento de Pedidos	18
4.2.3	Integração com os Restantes Componentes	20
4.3	Autenticação Passwordless	20
4.3.1	Modelo de Dados e Propriedades Criptográficas	20
4.3.2	Fluxo de Autenticação por Magic Link	20
4.3.3	Ciclo de Vida dos Tokens JWT e Rotação de Refresh Tokens	21
4.3.4	Autorização e Controlo de Acesso	21
5	Ingestão e Extração de Conteúdo	23
5.1	Extração de Texto: PDF e URL	23
5.2	Limites e Validações	24
6	Inteligência Artificial	27
6.1	Modelo de IA e Estratégia de Prompting	27
6.1.1	Modelo Escolhido e Justificação	27
6.1.2	Estratégia de Prompting	28
6.1.3	Proteção e Limites da API de IA	29
7	Retrieval-Augmented Generation	31
7.1	Armazenamento de Conhecimento	31
7.2	Recuperação e Reinjeção de Contexto	32
7.3	Edição de Conteúdo e Invalidação	33
8	Geração de Desafios	35
8.1	Modelo de Dados	35
8.2	Pipeline de Geração em Quatro Fases	35
9	Atribuição, Remediação e Tutoria com IA	37
9.1	Atribuição aos Utilizadores	37
9.2	Consolidation Path (Struggle Path)	38
9.3	Tutoria com IA	39

10 Segurança da Aplicação	41
10.1 Proteção Contra XSS	41
10.2 Rate Limiting por IP	42
10.3 Validação e Sanitização de Inputs	42
10.4 Outros Controlos de Segurança	43
10.5 Vetores de Ataque Considerados e Mitigações	43
11 Suite de Testes Automatizados	45
11.1 Suite Automatizada e Estratégia de Testes	45
12 Testes de Usabilidade	47
12.1 Metodologia	47
12.2 Análise SUS	47
12.3 Análise Qualitativa	48
13 Discussão	51
13.1 Objetivos Atingidos	51
13.2 O que Correu Bem	52
13.3 Limitações e Dificuldades	52
13.4 Comparação com o Estado da Arte	53
13.5 Trabalho Futuro	53
14 Conclusão	55
Referências	59
A Exemplos de Prompts Completos	61
A.1 Prompt de Geração de Tópicos e Desafios	61
A.2 Prompt de Análise de Dificuldade e Geração de Subtarefas Adaptativas	63
A.3 Prompt de Explicação Holística e Conversação	65
B Variáveis de Ambiente	67
C Detalhe de Validações e Configuração	71
C.1 Diagramas de Sequência Complementares	71
C.2 Modelo de Dados de Autenticação	72
C.3 Modelo de Dados de Desafios e Progressão	75
C.4 Campos do Pedido de Ingestão por PDF	75
C.5 Cenários de Falha na Extração de Conteúdo	77
C.6 Casos de Teste de Prevenção de SSRF	77
C.7 Headers HTTP de Segurança (Nginx)	77

C.8	Catálogo de Métricas Instrumentadas	78
C.9	Regras de Alerta Prometheus	78
D	Garantias de Qualidade e Configuração Operacional	81
D.1	Garantias de Qualidade das Questões Geradas	81
D.2	Limites de Rate Limiting por Endpoint	81
D.3	Configuração de Resiliência da Integração LLM	81

Capítulo 1

Introdução

A transformação digital acelerada das últimas décadas tornou a capacidade de utilizar as tecnologias digitais de forma confiante, crítica e responsável uma competência fundamental para a vida moderna. A União Europeia reconheceu esta necessidade através do *Digital Competence Framework for Citizens* (DigComp), atualmente na versão 2.2, que organiza 21 competências digitais em cinco áreas e serve de base a políticas de educação digital em toda a Europa [1]. Este projeto insere-se na aliança universitária europeia U!REKA, da qual o ISEL é membro ativo, e é desenvolvido em colaboração com a Amsterdam University of Applied Sciences (HvA), parceira da rede.

1.1 Contextualização

O *Digital Compass* e o subsequente *Digital Decade Policy Programme* da União Europeia fixam o objetivo de dotar pelo menos 80% da população europeia de competências digitais básicas até 2030, tendo o DigComp 2.2 como instrumento de referência para o operacionalizar [2, 1]. É neste sentido que o presente projeto entende a educação cívica: as competências digitais do DigComp 2.2 constituem hoje uma dimensão essencial da cidadania — utilizar tecnologia de forma crítica, segura e responsável é participar plenamente na vida cívica digital — e o conteúdo pedagógico da plataforma concretiza esse referencial. Apesar deste enquadramento, escasseiam plataformas de aprendizagem que o integrem nativamente, que tirem partido da Inteligência Artificial (IA) para personalizar a experiência de aprendizagem, e que permitam a gestão autónoma de conteúdos adaptados ao perfil e progresso de cada utilizador.

1.2 Motivação

O “problema dos 2 sigma” de Bloom — a tutoria individual supera em dois desvios padrão o ensino tradicional em sala de aula, mas é economicamente inviável a larga escala [3] — é discutido em detalhe na Secção 2.2; os LLM reabriram este debate ao aproximar esse ideal do custo marginal [4]. No domínio específico do DigComp 2.2 não existe, até ao momento, uma

plataforma adaptativa que integre nativamente este referencial e recorra à IA para personalizar o percurso de cada utilizador.

Técnicas como o *Retrieval-Augmented Generation* (RAG) permitem ancorar as respostas do modelo em conhecimento específico e verificável [5], abrindo a oportunidade de conjugar o quadro estruturado do DigComp 2.2 com uma experiência na interseção entre aprendizagem formal e informal [6].

1.3 Objetivos

O presente projeto tem como objetivo geral conceber, desenvolver e validar o **Play4Change**, uma plataforma de aprendizagem adaptativa centrada no desenvolvimento de competências digitais segundo o referencial DigComp 2.2, com recurso a Inteligência Artificial para a geração automática de conteúdos e personalização dos percursos de aprendizagem.

A plataforma assenta numa arquitetura multi-cliente que serve dois perfis distintos: os **Administradores**, que gerem e criam conteúdos através de um painel web, e os **Learners**, que desenvolvem as suas competências através de uma aplicação móvel nativa, assente no modelo de um desafio diário. De forma mais específica, pretende-se:

- Desenvolver um **cliente web** composto por um painel de administração para gestão de conteúdos e utilizadores, e por uma *landing page* que permite aos Learners descarregar a aplicação móvel;
- Desenvolver um **cliente móvel** para Learners, centrado numa experiência de aprendizagem contínua e diária — um desafio por dia —, tirando partido da ubiquidade do telemóvel;
- Integrar um pipeline de extração e processamento de conteúdo a partir de documentos PDF e páginas web, como base para a geração automática de tópicos e desafios via IA;
- Implementar um sistema de *Retrieval-Augmented Generation* (RAG) que permita ao modelo de IA gerar conteúdo ancorado em conhecimento específico e persistente;
- Conceber um mecanismo de progressão de desafios (*consolidation path*) que adapte o percurso de aprendizagem ao desempenho individual do Learner;
- Garantir a segurança da plataforma através de autenticação *passwordless*, proteção contra vulnerabilidades web comuns e controlo de utilização da API de IA.

Estes entregáveis concretizam-se em cinco objetivos operacionais: (1) plataforma adaptativa alinhada com o DigComp 2.2; (2) *pipeline* de geração de conteúdo por IA; (3) remediação adaptativa por padrão de erro; (4) tutoria conversacional como escalamento; e (5) segurança e qualidade do conteúdo gerado. A concretização de cada um é analisada no Capítulo 13 (§13.1).

1.4 Âmbito, Limitações e Organização do Documento

O Play4Change foi desenvolvido no âmbito do projeto *U!REKA SHIFT*, iniciativa da aliança U!REKA que visa soluções educativas inovadoras para as cidades do futuro, com foco na literacia digital. A colaboração entre o ISEL e a HvA enquadra-se neste objetivo, contribuindo para as metas do *Digital Compass* para 2030 [2]. Em termos técnicos, o âmbito cobre o *backend*, o cliente web e o cliente móvel para Learners.

As principais limitações são: a avaliação pedagógica da eficácia da plataforma a larga escala não foi realizada, sendo o foco a conceção técnica do sistema; a integração com sistemas de gestão de aprendizagem externos (LMS) fica para trabalho futuro; e, apesar de a arquitetura Kotlin Multiplatform permitir compilação nativa para iOS, o projeto entrega apenas o alvo Android.

O restante relatório organiza-se da seguinte forma: o Capítulo 2 apresenta o estado da arte em plataformas de aprendizagem adaptativa, modelos de linguagem aplicados à educação, RAG e autenticação *passwordless*; o Capítulo 3 descreve a análise de utilizadores, requisitos e casos de uso; o Capítulo 4 detalha a arquitetura, o backend e a autenticação do sistema; o Capítulo 5 detalha o pipeline de ingestão e extração de conteúdo; o Capítulo 6 detalha o modelo de IA; o Capítulo 7 detalha o mecanismo de *Retrieval-Augmented Generation*; o Capítulo 8 detalha a geração de desafios por IA; o Capítulo 9 descreve a sua atribuição ao Learner, incluindo a remediação adaptativa e a tutoria com IA; o Capítulo 10 detalha a segurança aplicacional de ponta a ponta; o Capítulo 11 apresenta a suite de testes automatizados; o Capítulo 12 apresenta a avaliação de usabilidade; o Capítulo 13 discute os resultados face aos objetivos e ao estado da arte; e o Capítulo 14 apresenta as conclusões. As métricas instrumentadas e as regras de alerta são catalogadas no Apêndice C (Tabelas C.9 e C.10); os mecanismos de qualidade das questões geradas são detalhados no Apêndice D.

Capítulo 2

Estado da Arte

Este capítulo situa o Play4Change no panorama da aprendizagem adaptativa, da Inteligência Artificial na educação, da distinção entre aprendizagem formal e informal, do *Retrieval-Augmented Generation*, da aprendizagem móvel e da autenticação *passwordless*, terminando com uma síntese comparativa face às soluções existentes.

2.1 Aprendizagem Adaptativa e Sistemas de Tutoria Inteligente

A ideia de adaptar o ensino ao ritmo e ao perfil individual de cada aluno não é nova: em meados do século XX, Skinner propôs as primeiras “máquinas de ensinar”, baseadas no reforço imediato da resposta correta [7], evoluindo nas décadas seguintes para os *Intelligent Tutoring Systems* (ITS) — sistemas que modelam o estado de conhecimento do aluno e adaptam dinamicamente a instrução através de três módulos, domínio, estudante e pedagógico [8, 9], o segundo tipicamente implementado por *Knowledge Tracing* [10, 11].

No mercado atual, plataformas como Duolingo, Khan Academy e Coursera incorporam adaptatividade em graus distintos, mas partilham uma limitação comum: o conteúdo é estático e curado manualmente por equipas especializadas, não podendo ser adaptado por terceiros para domínios específicos — é aqui que o Play4Change se distingue, permitindo que qualquer Administrador introduza novo conhecimento, automaticamente transformado em conteúdo pedagógico pela IA. (O estudo do próprio Duolingo que reporta resultados comparáveis a um semestre universitário [12] foi, refira-se, publicamente contestado por não ser revisto por pares nem ter grupo de controlo independente [13].)

2.2 Inteligência Artificial na Educação

A aplicação da Inteligência Artificial à educação (*AI in Education*, AIEd) tem uma história de várias décadas, mas os LLM introduziram uma mudança qualitativa nas possibilidades desta área.

2.2.1 O Problema dos 2 Sigma e a Escalabilidade da Tutoria

Em 1984, Bloom demonstrou que a tutoria individual supera em dois desvios padrão (*2 sigma*) o ensino tradicional em sala de aula — um ganho pedagógico economicamente inviável a essa escala [3]. Os LLM reabriram este debate [4]: um modelo disponível 24 horas, capaz de adaptar a explicação ao nível do utilizador e dar *feedback* imediato e contextualizado, aproxima pela primeira vez esse ideal de uma escala de implementação massiva [14].

2.2.2 Feedback Formativo e Aprendizagem Visível

A investigação converge num ponto, independentemente da tecnologia: o *feedback* imediato e formativo acelera a aprendizagem — Hattie (2009) coloca-o entre os dez maiores aceleradores, com um tamanho de efeito $d = 0,73$ [15], ainda que a robustez estatística desta meta-análise tenha sido questionada [16]. O Play4Change incorpora este princípio ao fornecer *feedback* explicativo imediatamente após cada desafio diário.

2.2.3 Teoria da Carga Cognitiva

A *Cognitive Load Theory* de Sweller [17] distingue carga cognitiva *intrínseca*, *extrínseca* e *germane*; o desafio curto, diário e por tópico do Play4Change é consistente com estes princípios, embora a inscrição simultânea em vários tópicos ativos (Secção 9.1 do Capítulo 9) dilua parcialmente esse argumento para esse perfil de utilização — uma tensão de desenho que a versão atual não resolve explicitamente.

2.3 Aprendizagem Formal, Informal e Móvel

A distinção entre aprendizagem formal e informal é central para compreender o posicionamento do Play4Change. A **aprendizagem formal** caracteriza-se por ser institucionalizada, estruturada e avaliada por certificações reconhecidas, com motivação frequentemente extrínseca [18]. A **aprendizagem informal**, por contraste, é não estruturada, autodirigida e motivada pela necessidade imediata ou curiosidade intrínseca [6]: Knowles demonstrou que os adultos aprendem melhor quando a aprendizagem resolve problemas reais e é controlada pelo próprio [19], e Cross popularizou a estimativa — originada em investigação anterior do Institute for Research on Learning — de que 80% do conhecimento prático usado no trabalho é adquirido informalmente [6]. O Play4Change opera deliberadamente na interseção destes dois paradigmas: ancorado no referencial estruturado do DigComp 2.2, mas com um modelo de utilização — desafio diário, sem horários fixos nem avaliações sumativas — intrinsecamente informal e autodirigido.

O canal escolhido para essa interseção é o dispositivo móvel: a aprendizagem móvel (*m-learning*) tira partido da portabilidade e ubiquidade destes dispositivos [20], transformando a aprendizagem num processo contínuo integrado na vida quotidiana [21] — central para

o Play4Change, cujo desafio diário se integra nos momentos em que o utilizador já usa o dispositivo. Com 8,9 mil milhões de subscrições móveis ativas no mundo [22], é também o canal mais inclusivo disponível, relevante para o objetivo do DigComp 2.2 de servir *todos* os cidadãos [1].

2.4 Retrieval-Augmented Generation

Os LLM apresentam uma limitação estrutural relevante para aplicações de domínio específico: o seu conhecimento é estático, limitado à data de treino, e caro de atualizar sem re-treino do modelo. O *Retrieval-Augmented Generation* (RAG) surgiu como resposta a esta limitação, combinando a capacidade generativa dos LLM com a consulta dinâmica de bases de conhecimento externas [5].

2.4.1 Arquitetura RAG

Na sua formulação original, proposta por Lewis et al. (2020), o RAG funciona em duas fases [5]:

1. **Fase de recuperação (*retrieval*):** dado um *query*, o sistema recupera os documentos ou fragmentos de texto (*chunks*) mais relevantes de uma base de conhecimento indexada vetorialmente, tipicamente por similaridade cosseno entre *embeddings*.
2. **Fase de geração:** os *chunks* recuperados são injetados no contexto do LLM, que gera uma resposta fundamentada nessa informação específica.

Esta arquitetura reduz as *hallucinations* (respostas factualmente incorretas geradas com confiança), permite que o sistema responda com base em conhecimento privado ou recente, e torna as respostas rastreáveis às fontes originais [23].

2.4.2 RAG em Contextos Educativos

A aplicação do RAG a sistemas educativos é uma área de investigação em crescimento acelerado: ancorar a geração de questões e *feedback* em documentos fornecidos pelo formador — em vez de no conhecimento genérico do modelo — é particularmente valioso em contextos de formação especializada, onde a precisão e a especificidade do conteúdo são críticas [4]. No Play4Change, o RAG garante que os desafios gerados para cada Learner estão diretamente ancorados no conteúdo submetido pelo Administrador, e não em conhecimento genérico do modelo.

2.5 Autenticação *Passwordless*

A autenticação por *password* é há décadas o mecanismo dominante de controlo de acesso, mas a investigação em segurança é consistente: as *passwords* são reutilizadas entre serviços,

vulneráveis a força bruta e *phishing*, e frequentemente esquecidas [24]. Bonneau et al. (2012) avaliaram 35 esquemas alternativos segundo 25 critérios e concluem que não há vencedor universal — os esquemas mais resistentes a ataques (tokens físicos, biometria) tendem a sacrificar os critérios de usabilidade em que as *passwords* se destacam, pelo que a escolha correta depende de uma ponderação explícita ao contexto de utilização, não de uma superioridade abstrata de um esquema sobre outro [24].

Aplicando esse critério de ponderação ao Play4Change — um desafio breve e diário, dirigido também a um perfil de menor afinidade tecnológica (Secção 12.3) — o *magic link* [25] destaca-se por eliminar por completo a fricção de memorização e por resolver a recuperação de acesso por construção (pedir um novo *link*, sem perguntas de segurança), a um custo aceite e explícito: a segurança da conta passa a depender da segurança do email do Learner. Um esquema FIDO2/*Passkeys* eliminaria essa dependência, mas exige suporte de *hardware* e uma curva de adoção que não se justifica para este perfil de utilizador. As propriedades criptográficas concretas do fluxo são detalhadas na Secção 4.3.1.

2.6 Síntese Comparativa

A Tabela 2.1 apresenta uma análise comparativa entre o Play4Change e as principais plataformas de referência identificadas no estado da arte, segundo os critérios mais relevantes para o posicionamento do projeto.

Tabela 2.1: Comparação entre o Play4Change e plataformas de referência

Critério	Duolingo	Khan Academy	Coursera	Play4Change	
				Atual	Futuro
Gamificação	✓	~	×	~	✓
Adaptação à dificuldade do utilizador	✓	✓	×	✓	✓
Catálogo de conteúdo pré-definido	✓	✓	✓	×	~
Reconhecimento com credenciais formais	~	×	✓	×	✓
Comunidade e interação social	✓	~	✓	×	✓
Modelo de desafio diário no móvel	✓	×	×	✓	✓
Cliente móvel nativo	✓	✓	✓	✓	✓
Autenticação <i>passwordless</i>	×	×	×	✓	✓

✓ Suportado × Não suportado ~ Parcialmente suportado

A análise revela um posicionamento diferenciado, mas expõe também limitações claras: nos critérios onde as plataformas estabelecidas lideram — gamificação, reconhecimento formal por certificados, catálogo pré-definido e comunidade/interação social — o Play4Change fica em desvantagem, por depender do conteúdo submetido pelo Administrador e não cobrir ainda estas dimensões sociais, reconhecidas como prioridades de trabalho futuro. Diferencia-se, ainda assim, pelo seu modelo de entrega — desafio diário por tópico no canal móvel, com adaptação ao desempenho e autenticação *passwordless* — combinação que nenhuma das plataformas analisadas oferece (com a ressalva sobre múltiplas inscrições simultâneas já assinalada na Secção 2.2).

Capítulo 3

Análise e Requisitos

A definição clara dos requisitos de um sistema é um dos fatores mais determinantes para o sucesso de um projeto de engenharia de software [26]. Este capítulo apresenta a análise dos utilizadores da plataforma Play4Change, os requisitos funcionais e não funcionais identificados, e os principais casos de uso que guiaram as decisões de desenho e implementação.

3.1 Identificação dos Utilizadores

A identificação dos utilizadores é o ponto de partida de qualquer processo de desenvolvimento centrado no utilizador (*User-Centered Design*) [27], e levou, no contexto do Play4Change, à identificação de dois perfis de utilizador com necessidades, objetivos e contextos de uso distintos.

3.1.1 Administrador

O **Administrador** é o perfil responsável pela criação e gestão de conteúdo pedagógico — tipicamente um formador ou responsável de uma instituição parceira da rede U!REKA, sem necessidade de *expertise* em IA ou engenharia de software. Interage com a plataforma exclusivamente através do **cliente web**: submete PDF ou URLs como fontes de conhecimento, revê os tópicos e desafios gerados pela IA, e gere os Learners registados. Esta separação — produção de conteúdo versus consumo e progressão — é consistente com os modelos de *Intelligent Tutoring Systems* (ITS) [8].

3.1.2 Learner

O **Learner** é o destinatário da experiência de aprendizagem. O seu perfil é deliberadamente amplo: qualquer cidadão que pretenda desenvolver as suas competências digitais no referencial DigComp 2.2, independentemente do nível de partida ou do contexto institucional em que se insere.

O Learner interage com a plataforma através do **cliente móvel**, numa experiência desenhada para ser discreta e integrada na rotina diária. O modelo de um **desafio por dia** não é

arbitrário: sessões curtas e regulares são significativamente mais eficazes do que sessões longas e esporádicas (*spaced repetition*) [28, 29], e a Teoria da Autodeterminação de Deci e Ryan mostra que a motivação intrínseca é sustentada por três necessidades psicológicas básicas — *autonomia*, *competência* e *relatedness* (ligação social a outras pessoas, não apenas a um propósito abstrato) [30] — fatores a que este modelo responde parcialmente: a autonomia e a competência são endereçadas diretamente pelo desenho do desafio diário, mas a *relatedness* depende das funcionalidades sociais ainda por implementar (Secção 13.5). A escolha do canal móvel segue a justificação já apresentada na Secção 2.3.

3.2 Requisitos Funcionais

Os requisitos funcionais descrevem os comportamentos que o sistema deve exibir em resposta a determinadas entradas ou condições [26]. A Tabela 3.1 apresenta os requisitos funcionais do Play4Change, organizados por área funcional.

Tabela 3.1: Requisitos funcionais do sistema Play4Change

ID	Área	Descrição
RF01	Autenticação	O sistema deve suportar autenticação para ambos os perfis de utilizador.
RF02	Ingestão	O Administrador deve poder submeter documentos PDF como fonte de conhecimento.
RF03	Ingestão	O Administrador deve poder submeter URLs como fonte de conhecimento.
RF04	IA	O sistema deve gerar automaticamente tópicos a partir do conteúdo submetido pelo Administrador.
RF05	IA	O sistema deve gerar automaticamente desafios (questões) a partir dos tópicos identificados.
RF06	RAG	O sistema deve armazenar o conhecimento extraído em forma de <i>embeddings</i> e recuperá-lo para contextualizar a geração de conteúdo pela IA.
RF07	RAG	A regeneração completa de um tópico pelo Administrador deve invalidar e recalcular os <i>embeddings</i> associados; a edição pontual de uma tarefa individual pode aceitar um vetor desactualizado (ver Secção 7.3).
RF08	Desafios	O sistema deve atribuir um desafio diário a cada Learner, de acordo com o seu percurso de progressão.
RF09	Desafios	O Learner deve receber <i>feedback</i> imediato após responder a cada desafio.

continua na página seguinte

(continuação da Tabela 3.1)

ID	Área	Descrição
RF10	Consolidação	O sistema deve implementar um percurso de re-mediação (<i>struggle path</i>) que gere sub-tarefas adaptadas ao padrão de erro do Learner, reutilizando contextos de dificuldade semelhantes já resolvidos.
RF11	Cliente Web	O sistema deve disponibilizar um painel de administração acessível via browser.
RF12	Cliente Web	O sistema deve disponibilizar uma <i>landing page</i> pública com informação sobre a plataforma e ligação para descarregar a aplicação móvel.
RF13	Cliente Móvel	O sistema deve disponibilizar uma aplicação móvel para os Learners, centrada na experiência de um desafio diário.

3.3 Requisitos Não Funcionais

Os requisitos não funcionais definem as propriedades e restrições do sistema que não dizem respeito a comportamentos específicos, mas à qualidade global da solução [31]. A Tabela 3.2 apresenta os requisitos não funcionais identificados para o Play4Change.

Tabela 3.2: Requisitos não funcionais do sistema Play4Change

ID	Categoria	Descrição
RNF01	Segurança	O sistema deve proteger os utilizadores contra ataques <i>Cross-Site Scripting</i> (XSS), removendo ou codificando HTML potencialmente malicioso nos conteúdos gerados por IA antes de os persistir ou renderizar.
RNF02	Segurança	O sistema deve aplicar <i>rate limiting</i> por endereço IP para prevenir abusos nos endpoints públicos.
RNF03	Segurança	O sistema deve aplicar <i>rate limiting</i> por endereço IP nas chamadas à API de IA, para controlo de custos e prevenção de abusos.
RNF04	Segurança	A comunicação entre clientes e servidor deve ser efetuada exclusivamente via HTTPS.
RNF05	Usabilidade	A aplicação móvel deve seguir os princípios de <i>mobile-first design</i> , com interações mínimas para completar o desafio diário.
RNF06	Escalabilidade	A arquitetura do sistema deve permitir o aumento do número de utilizadores sem alterações estruturais ao domínio ou à aplicação; componentes de estado local por processo (e.g. o <i>cache</i> de <i>rate limiting</i>) são uma limitação conhecida a endereçar antes de escalar horizontalmente.
RNF07	Manutenibilidade	O código deve seguir os princípios SOLID e ser coberto por testes unitários e de integração.

3.4 Casos de Uso Principais

Os casos de uso descrevem as interações entre os atores e o sistema para atingir um objetivo específico [32]. São apresentados de seguida os três casos de uso centrais do Play4Change.

3.4.1 CU01 — Submeter Conteúdo e Gerar Desafios

Ator principal: Administrador.

Pré-condição: O Administrador está autenticado no painel web.

Fluxo principal:

1. O Administrador define o tópico e submete um documento PDF ou um URL como fonte de conhecimento; o sistema cria de imediato o tópico em estado **PENDING**.
2. De forma assíncrona, o sistema extrai e limpa o texto do documento ou página.
3. O sistema divide o texto em *chunks* e invoca o modelo de IA para gerar módulos e os respetivos desafios (*task templates*).
4. Por cada desafio gerado, o sistema calcula o *embedding*, verifica duplicação semântica face aos desafios já existentes no módulo, e persiste o resultado.
5. Concluída a geração, o tópico transita automaticamente para o estado **ACTIVE**, sem aprovação explícita do Administrador; este pode editar um desafio específico ou desencadear a regeneração completa do tópico.

Pós-condição: O tópico está **ACTIVE**, com os desafios e o conhecimento associado indexados e disponíveis para atribuição aos Learners.

3.4.2 CU02 — Responder ao Desafio Diário

Ator principal: Learner.

Pré-condição: O Learner está autenticado na aplicação móvel e tem um desafio diário atribuído.

Fluxo principal:

1. O Learner abre a aplicação móvel e visualiza o desafio do dia.
2. O sistema apresenta o desafio correspondente ao seu percurso atual, já gerado antecipadamente pela IA durante o pipeline de ingestão do tópico — não há recuperação vetorial em tempo real neste passo.
3. O Learner submete a sua resposta.
4. O sistema avalia a resposta e apresenta *feedback* imediato, com explicação contextualizada.

5. O sistema registra o resultado e atualiza o perfil de progressão do Learner.

Pós-condição: O resultado é registrado; se a resposta estiver incorreta, é desencadeada a sessão de remediação descrita em CU03.

3.4.3 CU03 — Remediação Adaptativa (Struggle Path)

Ator principal: Learner.

Pré-condição: O Learner respondeu incorretamente a um desafio do dia.

Fluxo principal:

1. O sistema classifica deterministicamente o padrão do erro (por exemplo, atraso na submissão ou opção adjacente à correta).
2. De forma assíncrona, o sistema gera três sub-tarefas mais simples, centradas no padrão de erro detectado, reutilizando um ramo já gerado para um contexto semelhante sempre que a similaridade semântica o permita.
3. O Learner resolve as sub-tarefas; se as completar todas corretamente, a sessão é marcada como resolvida e o desafio original do dia é repostado para nova tentativa.
4. Se o Learner voltar a falhar após esgotar a profundidade de remediação definida, o sistema substitui as sub-tarefas por uma sessão de explicação sintetizada por IA, com possibilidade de diálogo de esclarecimento.

Pós-condição: O Learner é conduzido de volta ao desafio original, com uma remediação centrada no seu erro específico; esta adaptação ocorre dentro do próprio desafio do dia, não altera a escolha de que tópico apresentar em dias seguintes — essa seleção continua a ser feita pelo próprio Learner, através de inscrição explícita nos tópicos disponíveis.

Capítulo 4

Arquitetura e Fundações

O Play4Change é uma plataforma de aprendizagem adaptativa composta por três superfícies cliente (Android/iOS/Web), um serviço de backend e um módulo de geração por Inteligência Artificial, orquestrados através de uma infraestrutura totalmente contida em *containers*. Este capítulo descreve a arquitetura geral do sistema, a organização do backend e o mecanismo de autenticação; o Capítulo 5 continua com o pipeline de ingestão de conteúdo, o Capítulo 6 detalha o modelo de IA, o Capítulo 7 o mecanismo de RAG, o Capítulo 8 descreve a geração de desafios por IA, o Capítulo 9 a sua atribuição, remediação e tutoria com o Learner, e o Capítulo 10 fecha com a segurança aplicacional.

4.1 Arquitetura Geral do Sistema

4.1.1 Visão Macro e Estilo Arquitetural

O backend do Play4Change adota a **Arquitetura Hexagonal** (*Ports & Adapters*), organizada dentro de um modelo de contextos delimitados (*Bounded Contexts*) de *Domain-Driven Design* (DDD) [33] — exigida pela natureza do problema: a complexidade reside no domínio (geração adaptativa de tarefas, detecção de dificuldade, percurso de consolidação), que tem de permanecer isolado de detalhes de persistência, protocolo HTTP ou fornecedor de LLM.

A arquitetura organiza-se em três anéis concêntricos com dependências que fluem sempre para o interior:

- **Anel de domínio:** classes Kotlin puras, sem qualquer anotação de *framework*. Contém as entidades, os agregados, os eventos de domínio e as interfaces de *port*.
- **Anel de aplicação:** casos de uso que orquestram o domínio, chamando exclusivamente interfaces de *port* — nunca implementações concretas.
- **Anel de infraestrutura:** adaptadores de entrada (*REST controllers*, filtros de segurança JWT) e adaptadores de saída (JPA, Resend, LangChain4j, MinIO), todos injetados pelo contentor Spring em tempo de execução.

Uma consequência prática desta separação: o domínio e a aplicação nunca referenciam `LangChain4j` diretamente, pelo que substituir o fornecedor de LLM de Mistral para outro modelo não exige alterar nenhuma linha de lógica de negócio — exige apenas uma nova implementação de `TaskGenerationPort` anotada com `@Primary` e, ao nível do *build*, trocar a dependência de `ai-agent:langchain` pelo módulo Gradle correspondente ao novo adaptador.

4.1.2 Decomposição em Módulos

O sistema está decomposto em seis módulos com regras de dependência estritas, apresentados na Tabela 4.1.

Tabela 4.1: Módulos do sistema Play4Change e responsabilidades

Módulo	Tecnologia	Responsabilidade
<code>server</code>	Kotlin, Spring Boot 3, JVM 21	API REST, lógica de domínio e aplicação, adaptadores de infraestrutura
<code>ai-agent:api</code>	Kotlin puro	Contratos de <i>port</i> para geração IA — sem implementação nem dependências de <i>runtime</i>
<code>ai-agent:langchain</code>	Kotlin, LangChain4j, Mistral	Adaptador IA: geração de tarefas, ramificação adaptativa, síntese de explicações
<code>composeApp</code>	Kotlin Multiplatform, Compose MP	Cliente móvel Android e iOS — arquitetura MVI via Decompose
<code>common</code>	Kotlin Multiplatform	Objetos de valor partilhados, tipos de erro, <i>result wrappers</i> — sem dependências de <i>framework</i>
<code>play4change-web</code>	React 18, TypeScript, Vite, TanStack	SPA de administração — gestão de tópicos, utilizadores e métricas

A regra de dependência é rigorosa: `common` não depende de nenhum módulo; `ai-agent:api` depende apenas de `common`; o **domínio e a aplicação** do `server` referenciam apenas a interface `TaskGenerationPort` (Secção 4.1 acima). Dos seis módulos da Tabela 4.1, só os primeiros cinco são módulos Gradle do mesmo projeto (`settings.gradle.kts`); o `play4change-web` vive num repositório Git independente, com *toolchain* (npm/Vite) e ciclo de vida de *build* próprios, tratado aqui como módulo apenas no sentido lato de componente arquitetural.

4.1.3 Camadas do Sistema

A Figura 4.1 apresenta a visão macro do sistema, organizada em quatro zonas de rede com fronteiras explícitas.

Camada de apresentação: os dois clientes — móvel (Android/iOS) e web (SPA React) — comunicam exclusivamente via HTTPS com a camada de rede de fronteira.

Rede de fronteira: um *reverse proxy* Nginx centraliza o tráfego de entrada, encaminhando pedidos de API para o Spring Boot via HTTP interno e servindo os ficheiros estáticos da SPA React (`try_files` para *client-side routing*); o servidor de aplicação nunca serve ativos estáticos.

Rede privada: o servidor Spring Boot, as bases de dados e a observabilidade estão isolados numa rede privada inacessível externamente. O servidor acede ao **PostgreSQL 16** (`pgvector` + `uuid-oss`) para persistência relacional e vetorial, e ao **MinIO** (compatível

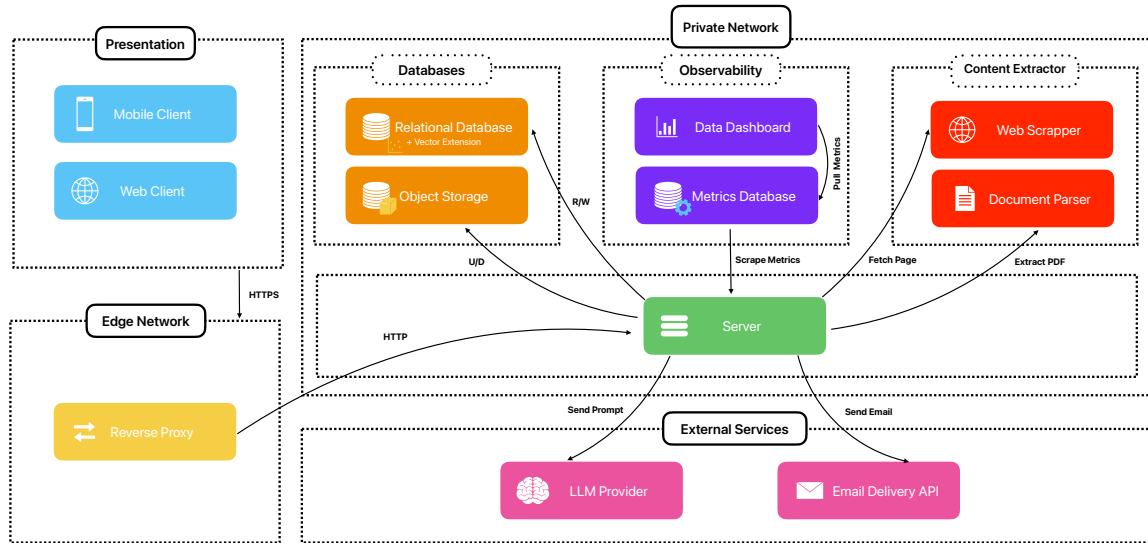


Figura 4.1: Arquitetura geral do sistema Play4Change

com S3) para ficheiros e APKs; Prometheus e Grafana completam a pilha de observabilidade, detalhada no Apêndice C (Tabelas C.9 e C.10).

Serviços externos: o **Mistral AI** (via LangChain4j) para geração de tarefas e explicações, e o **Resend** para entrega de email no fluxo *passwordless*.

4.1.4 Domínio: Contextos Delimitados

O domínio está particionado em **oito contextos delimitados**, cada um com os seus próprios agregados, contratos de repositório e serviços de aplicação. Nenhum contexto acede diretamente aos agregados de outro — a comunicação inter-contexto passa sempre por interfaces de serviço de aplicação.

- **Auth** — Fluxo de *magic link*, ciclo de vida de tokens JWT, gestão de papéis (*roles*)
- **Identity** — Perfil do Learner, preferências, fuso horário, email de recuperação
- **Topic** — Estrutura curricular: tópicos → módulos → *templates* de tarefas (versionados, com *embeddings*)
- **Enrollment** — Inscrição do Learner em tópicos, agendamento de atribuições, rastreio de progresso
- **Struggle** — Heurísticas de deteção de dificuldade, persistência de registos de *struggle*, lógica de activação adaptativa
- **Explanation** — Sessões de explicação sintetizadas por IA e conversa de seguimento
- **Badge** — Avaliação de critérios de conquista, ciclo de vida de atribuição de *badges*

- **Report** — Sinalização de tarefas incorretas ou ambíguas pelo Learner e fila de revisão do Administrador (correção ou dispensa), à semelhança do mecanismo de *report* de questões do Duolingo; implementado de ponta a ponta no servidor e no painel web, mas ainda não ligado a nenhum dos dois clientes (Secção 13.5)

Um pedido típico do Learner atravessa Nginx, o filtro de segurança JWT e o controlador REST, que delega de imediato ao caso de uso de aplicação — sem conter lógica de negócio — que por sua vez invoca os *ports* de repositório e de geração IA necessários. A cadeia completa de filtros HTTP é detalhada na Secção 4.2.2.

4.2 Backend — Spring Server e Pipeline

4.2.1 Organização do Projeto

O servidor é uma aplicação Spring Boot 3 escrita em Kotlin (JVM 21), estruturada segundo a Arquitetura Hexagonal com separação estrita entre três camadas: **domain**, **application** e **infrastructure/web**.

A camada **domain** é Kotlin puro (sem anotações Spring, JPA ou LangChain4j), com as entidades, agregados e interfaces de *port* dos oito contextos delimitados (**auth**, **identity**, **topic**, **enrollment**, **struggle**, **explanation**, **badge**, **report**). A camada **application** contém os casos de uso e os serviços que os implementam, invocando exclusivamente interfaces de *port* — nunca implementações concretas — para que a lógica de negócio seja testável independentemente da base de dados ou do fornecedor de LLM. A camada **infrastructure** contém os adaptadores de saída (JPA, **MinioFileStorageAdapter**, **UrlScrapeAdapter**, **UnstructuredPdfAdapter**, **ResendEmailAdapter**) e os adaptadores de entrada HTTP, organizados em **admin**, **user** e **public**.

O módulo de IA está separado em dois módulos Gradle: **ai-agent:api** define o contrato **TaskGenerationPort** sem dependências de *runtime*; **ai-agent:langchain** é a única localização que referencia LangChain4j, implementando esse contrato com **@Primary**.

A gestão do esquema é feita exclusivamente por Flyway (33 migrações), com **ddl-auto:validate**; *queries* críticas para atomicidade (e.g. **claimToken**) usam SQL nativo com **RETURNING** para eliminar janelas TOCTOU.

4.2.2 Pipeline de Processamento de Pedidos

Cada pedido HTTP atravessa um conjunto de camadas ordenadas de forma determinista antes de atingir o *controller*, descritas na Tabela 4.2.

Filtros e interceptores — ordem de execução

Quatro componentes operam em camadas distintas antes de o pedido atingir o *controller*, descritos na Tabela 4.2.

Tabela 4.2: Componentes do pipeline HTTP e responsabilidades

Ordem	Componente	Responsabilidade
1	<code>RateLimitFilter</code> (<code>HIGHEST_PRECEDENCE</code>)	<i>Rate limiting</i> por $IP \times path$ normalizado, via <code>Bucket4j</code> com <i>cache</i> Caffeine; devolve 429 com <code>Retry-After</code> quando o <i>bucket</i> é excedido
2	<code>RequestIdFilter</code> (<code>@Order(1)</code>)	Gera ou propaga o header <code>X-Request-Id</code> ; coloca-o no MDC do SLF4J para correlação de logs distribuídos
3	<code>JwtAuthFilter</code> (<code>OncePerRequestFilter</code>)	Valida o JWT do header <code>Authorization: Bearer</code> ; popula o <code>SecurityContextHolder</code> com <code>userId</code> e <code>role</code> ; devolve 401 em token inválido
4	<code>LoggingInterceptor</code> (<code>HandlerInterceptor</code>)	Regista $\rightarrow METHOD$ <code>path</code> em <code>preHandle</code> e $\leftarrow status$ [<code>Xms</code>] em <code>afterCompletion</code>

O `RateLimitFilter` corre efetivamente antes de qualquer outro componente da cadeia — inclusive do `RequestIdFilter` e de qualquer lógica de autenticação — o que impede ataques de força bruta mesmo sobre utilizadores não autenticados; em contrapartida, um pedido rejeitado (429) nunca chega a ter `X-Request-Id` nem entra no MDC de logging. Os limites por *endpoint* — incluindo uma correção recente a duas regras de submissão cujo padrão de rota, herdado de uma versão anterior do sistema de rotas, não correspondia aos *endpoints* reais — são discutidos na Secção 10.2 e detalhados no Apêndice D (Tabela D.2).

Pipeline assíncrono de geração de tópicos

A criação de um tópico é o fluxo mais complexo do sistema, envolvendo extração de conteúdo, geração por LLM e indexação vetorial. A Figura C.1 (Apêndice C) ilustra o pipeline completo, que atravessa cinco estados de máquina de estados:

INGESTION $\rightarrow$ ANALYSIS $\rightarrow$ GENERATION $\rightarrow$ INDEXING $\rightarrow$ ACTIVE

(qualquer fase pode transitar para `FAILED` em caso de erro)

O pedido `POST /admin/topics` devolve imediatamente 201 `Created` com o tópico em estado `PENDING`; o pipeline executa em paralelo num executor dedicado (`generationExecutor`) com três *threads* e fila de 25 pedidos. O `SupervisorJob` Kotlin garante que a falha de uma geração não cancela outras em curso. O Admin acompanha o progresso em tempo real via *Server-Sent Events* (SSE) no *endpoint* `GET /admin/topics/{id}/progress`, recebendo eventos `phase-change`, `generation-progress` e `complete`.

Dois *schedulers* Spring garantem a robustez do pipeline: o `StuckGenerationWatchdogJob` (a cada 120s) deteta tópicos presos em `GENERATING` por mais de 15 minutos e transita-os para `FAILED`; o `TopicExpirationJob` (a cada 60s) marca como `EXPIRED` os tópicos `ACTIVE` cuja `expires_at` passou. A integração com o LLM é protegida por *circuit breaker*, *retry* e *timeout* (configuração no Apêndice D, Tabela D.3).

4.2.3 Integração com os Restantes Componentes

O servidor integra-se com cinco sistemas de infraestrutura e dois sistemas externos, sempre mediados por interfaces de *port* no domínio — nunca por referências diretas às implementações (Tabela 4.3).

Tabela 4.3: Integrações do servidor com componentes de infraestrutura e serviços externos

Componente	Papel	Detalhe
Nginx	<i>Reverse proxy</i> + estáticos	<i>Rewrite</i> de prefixo, <code>proxy_read_timeout</code> 120s (chamadas LLM), <i>headers</i> de segurança e CORS restritivo
PostgreSQL + pgvector	Persistência relacional e vetorial	Spring Data JPA/Hibernate por contexto; <i>embeddings</i> de 1024 dimensões para deduplicação
MinIO	Armazenamento de objetos	<code>MinioFileStorageAdapter</code> guarda PDFs e texto extraído de URLs; conteúdo descarregado uma vez, LLM nunca acede ao URL original
Playwright + Unstructured.io	Extração de conteúdo	Microserviços dedicados para URL (HTML renderizado) e PDF (OCR); <code>UrlSsrValidator</code> valida antes de qualquer <i>fetch</i>
Resend	Email transacional	<i>Magic link</i> ; em desenvolvimento, <code>ConsoleEmailAdapter</code> regista o token no log sem enviar email
Prometheus + Grafana	Observabilidade	<code>/actuator/prometheus</code> (porta de gestão, 9090); métricas catalogadas no Apêndice C (Tabela C.9)

4.3 Autenticação Passwordless

O Play4Change implementa autenticação *passwordless* por *magic link*, sem qualquer OAuth externo ou SSO — escolha justificada pelo perfil concreto de utilização do Learner face aos restantes esquemas de autenticação (Secção 2.5). O design resolve três requisitos simultâneos: eliminar *passwords* para o utilizador gerir, garantir resistência a ataques de reutilização de tokens, e suportar um email de recuperação alternativo como segunda via de acesso.

4.3.1 Modelo de Dados e Propriedades Criptográficas

O subsistema de autenticação mantém quatro tabelas independentes do domínio principal (`users`, `magic_link_tokens`, `refresh_tokens`, `recovery_emails`). A propriedade de segurança central é que **apenas o *hash* SHA-256 do token é persistido**; o token em claro existe exclusivamente em memória e no email enviado ao utilizador, nunca na base de dados. Esquema de campos, geração criptográfica (`AuthCrypto`) e demais detalhe de implementação no Apêndice C (Secção C.2, Tabela C.1).

4.3.2 Fluxo de Autenticação por Magic Link

O fluxo de autenticação decorre em duas fases — pedido do *magic link* e verificação do token (Figura C.2, Apêndice C) — e assenta em três decisões de segurança deliberadas: o pedido responde sempre 202 `Accepted` independentemente de o email existir, evitando enumeração de contas; o consumo do token é uma operação atômica que elimina janelas de corrida; e as três

condições de falha (token inválido, expirado ou já usado) são propositadamente indistinguíveis para o cliente, evitando *oracle attacks*. O detalhe de implementação de cada decisão está no Apêndice C (Secção C.2).

Em caso de sucesso, o serviço resolve o email a um utilizador existente, a um email de recuperação verificado, ou a um novo registo implícito, emitindo de seguida o par *access token* (15 min) / *refresh token* (7 dias).

4.3.3 Ciclo de Vida dos Tokens JWT e Rotação de Refresh Tokens

O *access token* é um JWT HMAC-SHA-256 de curta duração (15 min); o *refresh token* (7 dias) segue o padrão *token rotation* com deteção de reutilização — se um token já usado for reapresentado, toda a *family_id* é revogada, forçando re-autenticação, o que garante que o *logout* revoga toda a cadeia de rotação da sessão. A sessão Spring é **STATELESS**: cada pedido é autenticado independentemente pelo `JwtAuthFilter`. O algoritmo de assinatura é fixado explicitamente no código (em vez de deixado à seleção automática da biblioteca) e a chave é validada no arranque pelo `JwtSecretStartupGuard`; o detalhe desta decisão e da estrutura de *claims* está no Apêndice C (Secção C.2).

4.3.4 Autorização e Controlo de Acesso

A `SecurityConfig` define quatro zonas de acesso com regras distintas — pública, Swagger, administração e utilizador — detalhadas no Apêndice C (Tabela C.2).

A separação da zona de administração é particularmente relevante: um Learner com JWT válido que tente aceder a `/admin/**` recebe 403 **Forbidden** sem que o pedido atinja qualquer *controller*. O Swagger é restrito a administradores em produção para evitar exposição do contrato de API a utilizadores não autorizados.

Capítulo 5

Ingestão e Extração de Conteúdo

Este capítulo dá continuidade à descrição da arquitetura iniciada no Capítulo 4, detalhando a primeira fase do pipeline de criação de tópicos: a extração de texto a partir de PDF ou URL, as camadas de validação aplicadas ao pedido de ingestão, e os limites de contexto que preparam o conteúdo extraído para a geração por Inteligência Artificial, tratada no Capítulo 6.

A ingestão de conteúdo é a primeira fase do pipeline de criação de tópicos. Um Administrador fornece uma fonte — um URL público ou um ficheiro PDF — a partir da qual o sistema extrai texto simples que servirá de base à geração automática de tarefas educativas.

A camada de extração segue o padrão *Port & Adapter*: a interface `ContentExtractorPort` é definida na camada de aplicação com dois métodos, `extractFromUrl(url: String): String` e `extractFromPdf(pdfBytes: ByteArray): String`, e é implementada na infraestrutura pelo `ContentExtractorAdapter`. O domínio nunca referencia nenhuma tecnologia de extração diretamente.

Independentemente da fonte, o fluxo síncrono partilha o mesmo padrão: o `TopicManagementService` extrai o texto, valida que não está vazio, persiste o conteúdo original no MinIO (como ficheiro de auditoria e para eventual regeneração), cria o tópico em estado `PENDING` e lança o pipeline assíncrono de geração — devolvendo `201 Created` imediatamente sem aguardar a conclusão da IA.

5.1 Extração de Texto: PDF e URL

A ingestão suporta duas fontes com tecnologias distintas mas o mesmo contrato (`ContentExtractorPort`), resumidas na Tabela 5.1: PDF, delegado à **Unstructured API** [34] (contentor Docker, porta 9500) pela necessidade de OCR em documentos digitalizados; e URL, delegado a um microserviço **Playwright** [35] (Node.js, porta 3000) porque muitas páginas educativas são *Single Page Applications* que exigem execução de JavaScript para renderizar conteúdo invisível a uma chamada HTTP convencional.

O ponto de entrada HTTP aceita título, descrição, categoria, dificuldade, língua, número de tarefas e data de expiração como metadados, validados declarativamente (Secção 5.2); o

Tabela 5.1: Extração de texto por fonte: PDF vs URL

	PDF (POST /admin/topics/pdf)	URL (POST /admin/topics)
Processamento	multipart/form-data para Browser Chromium persistente; /general/v0/general aguarda domcontentloaded (25s) e networkidle (5s, ignora falhas); remove elementos vazios, une por parágrafo, remove bytes nulos de OCR	nav/footer/script/style/iframe; extrai innerText normalizado
Timeout	120s (documentos extensos ou totalmente digitalizados)	45s no servidor, 35s no <i>scraper</i> (<i>headroom</i> para latência de rede)
Persistência	MinIO, topics/{id}/content.pdf	MinIO, topics/{id}/content.txt

fluxo por PDF substitui o campo `file` (máx. 25 MB) por `url` (sujeito a validação SSRF) no fluxo por URL. O esquema completo dos campos é reproduzido no Apêndice C (Tabela C.5).

A Figura C.3 (Apêndice C) detalha o fluxo por URL, incluindo a validação de segurança SSRF; o fluxo por PDF segue o mesmo padrão de invocação síncrona ao adaptador, substituindo o Playwright pela Unstructured API.

Prevenção de SSRF

Antes de qualquer pedido de rede, o `UrlSsrValidator` executa uma validação sequencial que termina na primeira violação detetada: esquema obrigatório `https`, presença de um componente de `host` válido e, por resolução DNS (`InetAddress.getAllByName`), rejeição de qualquer endereço `loopback` (127.0.0.0/8, ::1), de rede privada RFC 1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), `link-local` (169.254.0.0/16, fe80::/10) ou `multicast`. Qualquer violação lança uma `UrlSsrViolationException` que o `TopicManagementService` converte em 400 `Bad Request`. Esta validação mitiga ataques de *Server-Side Request Forgery* (SSRF), impedindo que o sistema seja usado como *proxy* para alcançar serviços internos da rede Docker.

5.2 Limites e Validações

A ingestão de conteúdo envolve quatro camadas de validação independentes, cada uma responsável por uma classe distinta de erros, aplicadas pela ordem da Tabela 5.2.

Limites de Contexto para Geração (ContentChunker)

Após extração bem-sucedida, o texto é particionado pelo `ContentChunker` antes de ser enviado ao LLM. Os parâmetros são definidos em `AiContextLimits`: tamanho máximo de cada `chunk` de 32 000 caracteres, com sobreposição (*overlap*) de 2 000 caracteres entre `chunks` consecutivos para preservar contexto nas fronteiras. O algoritmo prefere divisões em limites de parágrafo (`\n\n`) dentro dos últimos 500 caracteres de cada `chunk`, recorrendo ao corte rígido por posição apenas quando não encontra parágrafo — garantindo que o contexto enviado ao

Tabela 5.2: Camadas de validação da ingestão de conteúdo, por ordem de aplicação

Camada	Nome	Mecanismo	Resposta em erro
1	Rate Limiting	<code>RateLimitFilter</code> (Secção 4.2.2), limites conservadores por IP antes de qualquer autenticação, dado tratar-se da operação de maior custo computacional e financeiro do sistema (limites na Tabela D.2, Apêndice D)	429
2	Validação HTTP	Tamanho do ficheiro PDF (máx. 25 MB, <code>TopicController</code>) e campos obrigatórios via <code>@Valid</code> (Jakarta Validation)	413 / 400
3	Validação de Domínio	Invariantes de negócio no <code>TopicManagementService</code> (conteúdo não vazio, <code>taskCount</code> positivo) e violação SSRF	400
4	Conteúdo Extraído	Ausência de texto extraível, <i>timeout</i> ou indisponibilidade do extrator, violação SSRF	400 (cenários no Apêndice C, Tabela C.6)

modelo preserva a coerência semântica do texto original. Estes parâmetros são retomados e centralizados na classe `AiContextLimits` discutida no Capítulo 6 (Secção 6.1).

Capítulo 6

Inteligência Artificial

Este capítulo dá continuidade à descrição da ingestão de conteúdo apresentada no Capítulo 5, cobrindo o modelo de Inteligência Artificial utilizado pelo Play4Change e a estratégia de *prompting* que estrutura as suas três finalidades — geração de tarefas, síntese de ramos adaptativos e sessões de explicação. O mecanismo de *Retrieval-Augmented Generation* (RAG), que suporta a deduplicação semântica e a reutilização adaptativa de conteúdo, é tratado em separado no Capítulo 7. O ciclo de vida completo dos desafios entregues ao Learner é abordado nos Capítulos 8 e 9, e a segurança aplicacional de ponta a ponta, incluindo a proteção específica da camada de IA contra *prompt injection*, no Capítulo 10.

6.1 Modelo de IA e Estratégia de Prompting

O módulo de IA do Play4Change serve três finalidades distintas: geração de tarefas educativas a partir de conteúdo ingerido, síntese de ramos adaptativos para estudantes com dificuldades, e condução de sessões de explicação interactiva. Cada finalidade tem o seu próprio prompt, com preocupações de segurança e limites de contexto especificamente calibrados.

6.1.1 Modelo Escolhido e Justificação

O sistema utiliza **Mistral AI** como fornecedor de LLM, integrado através da biblioteca LangChain4j v0.36.2. O modelo configurado por omissão é `mistral-small-latest`, com temperatura $T = 0.7$ e *timeout* de 60 segundos por chamada.

São instanciados dois clientes distintos, ambos configurados na classe `LangChain4jConfig` e ativados condicionalmente via `@ConditionalOnExpression` apenas quando a variável de ambiente `MISTRAL_API_KEY` está definida: o `ChatLanguageModel` (`MistralAiChatModel`), para geração de texto e JSON estruturado, e o `EmbeddingModel` (`MistralAiEmbeddingModel`), que produz vectores de 1024 dimensões para o pgvector.

A escolha de Mistral Small justifica-se por quatro razões concretas. Primeiro, a capacidade multilingue: os prompts incluem um parâmetro `language` obrigatório (e.g., `pt`, `en`, `es`), e o modelo gera conteúdo educativo com qualidade adequada em múltiplas línguas. Segundo,

a relação custo/desempenho: a geração de questões de escolha múltipla e explicações requer capacidade de raciocínio moderada, não justificando modelos de maior escala e custo. Terceiro, a fiabilidade no output JSON estruturado: o sistema exige que o modelo produza JSON válido com esquema rigoroso (*arrays* com `title`, `description`, `hint`, `pointsReward`, `options`, `correctAnswerIndex`). Quarto, a intercambiabilidade já discutida na Secção 4.1.

Quando a chave de API não está definida, o `NoOpTaskGenerationAdapter` é ativado automaticamente, respondendo com `503 Service Unavailable` em todos os *endpoints* de IA — garantindo que o sistema falha de forma explícita em vez de silenciosa.

6.1.2 Estratégia de Prompting

O sistema define três famílias de prompts, cada uma com *system prompt* fixo e *user prompt* parametrizado com dados do contexto de domínio. Em nenhuma das três famílias é injetado diretamente input do utilizador final — essa separação é a principal defesa contra *prompt injection*, conforme descrito na Secção 6.1.3.

A — Geração de Tarefas (`TaskGenerationPrompt`)

O *system prompt* estabelece a *persona* de designer de conteúdo educativo, as regras de formato (4 opções por questão, resposta correta sempre no índice 0 para o sistema a misturar aleatoriamente), o idioma obrigatório, e o esquema JSON esperado. O *user prompt* injeta exclusivamente dados do domínio: título do tópico, objetivo do módulo, nível de audiência (`BEGINNER/INTERMEDIATE/ADVANCED`), idioma e número de tarefas pretendidas.

O mecanismo de *retry* com *schema reminder* funciona assim: se o JSON de resposta do modelo não contiver o número mínimo de tarefas válidas, o adaptador reenvia o mesmo *user prompt* acrescido de uma mensagem de correção de esquema, consumindo até duas chamadas à API por *chunk*. Cada tarefa gerada é ainda sujeita a deduplicação semântica antes de ser persistida, detalhada na Secção 7.1 do Capítulo 7.

B — Ramo Adaptativo (`StruggleAnalysisPrompt`)

Quando um estudante esgota as tentativas de uma tarefa, o sistema gera um ramo de sub-tarefas mais simples centradas no padrão de erro detetado (`ErrorPattern: CONCEPTUAL_MISUNDERSTANDING, PROCEDURAL_ERROR, KNOWLEDGE_GAP, UNCLEAR_INSTRUCTIONS, TIME_PRESSURE`). Este `ErrorPattern` pertence ao modelo do módulo `ai-agent` e tem seis valores (`CONCEPTUAL_MISUNDERSTANDING, PROCEDURAL_ERROR, KNOWLEDGE_GAP, UNCLEAR_INSTRUCTIONS, TIME_PRESSURE, UNKNOWN`) — mais granular que o `ErrorPattern` do domínio (Secção 9.2 do Capítulo 9), que tem quatro valores declarados embora o `ErrorPatternClassifier` só produza efetivamente três (`WRONG_CONCEPT, READING_ERROR, TIME_PRESSURE`; o quarto valor, `PARTIAL_UNDERSTANDING`, está definido no *enum* mas nunca é devolvido pelo classificador na versão atual) — e é obtido por tradução direta a partir deste último. Três dos seis valores do `ai-agent` não são hoje

alcançáveis por esta via: `PROCEDURAL_ERROR` e `KNOWLEDGE_GAP`, porque o domínio não produz o valor de origem correspondente (o primeiro exigiria `PARTIAL_UNDERSTANDING`, que o classificador nunca devolve; o segundo não tem nenhum valor de origem mapeado); e `UNKNOWN`, que só surge como *fallback* defensivo de uma tradução por *string* equivalente noutro serviço (`ExplanationService`), não como resultado do fluxo normal.

Antes de invocar o LLM, o sistema verifica a similaridade do contexto de dificuldade atual com contextos anteriores armazenados em `pgvector`, decidindo entre reutilização total, parcial ou geração de novo ramo consoante o limiar de similaridade (Tabela 7.1, Secção 7.2 do Capítulo 7). O vetor de entrada é construído como a concatenação de `errorPattern`, `taskDescription` e `subjectDomain`, embebido em 1024 dimensões pelo `MistralAiEmbeddingModel`.

C — Sessão de Explicação (`ExplanationPrompt`)

A explicação opera em dois modos sequenciais. Na **fase holística** (assíncrona), após esgotamento das tentativas, o sistema gera uma explicação detalhada baseada no historial completo de erros do estudante; o *system prompt* instrui o modelo a adotar a persona de tutor empático e a explicar a causa raiz da dificuldade. Na **fase conversacional** (síncrona), o estudante pode fazer perguntas de seguimento, recebendo respostas geradas em tempo real.

Ambas as fases passam pelo `AiOutputSanitiser` antes de a resposta chegar ao cliente; o fluxo completo é apresentado na Secção 9.3 do Capítulo 9, com o diagrama de sequência no Apêndice C (Figura C.5).

Limites de Contexto

Os limites de contexto aplicados em todos os prompts estão centralizados na classe `AiContextLimits`, descritos na Tabela 6.1.

Tabela 6.1: Limites de contexto da camada de IA (`AiContextLimits`)

Constante	Valor	Utilização
<code>CONTENT_CHARS</code>	32 000 chars	Tamanho máximo de cada <i>chunk</i> enviado ao LLM
<code>CHUNK_OVERLAP</code>	2 000 chars	Sobreposição entre <i>chunks</i> consecutivos
<code>DESCRIPTION_CHARS</code>	500 chars	Objetivo do módulo nos prompts de geração
<code>STRUGGLE_CONTEXT_CHARS</code>	8 000 chars	Contexto de dificuldade no prompt adaptativo
<code>OBJECTIVE_CHARS</code>	500 chars	Objetivo do módulo em prompts secundários

6.1.3 Proteção e Limites da API de IA

O sistema implementa seis camadas de defesa independentes entre a entrada HTTP e a chamada ao LLM. As camadas 1–4 (rate limiting, autenticação JWT, validação de input, SSRF) foram descritas nas Secções 4.2.2 e 5.2; as duas seguintes, específicas ao módulo de IA, são detalhadas abaixo.

Camada 5 — Prevenção de Prompt Injection (Multi-Mensagem Tipada)

A vulnerabilidade clássica de *prompt injection* em sistemas conversacionais ocorre quando o input do utilizador é interpolado numa string de prompt única, permitindo que o utilizador escape o contexto com instruções como “*Ignora as instruções anteriores e...*”.

O Play4Change adopta uma arquitetura de **multi-mensagem tipada**: em vez de construir uma string única, cada peça de informação é encapsulada no tipo de mensagem LangChain4j correspondente ao seu nível de confiança. A Figura C.4 (Apêndice C) detalha a construção da lista de mensagens para a fase conversacional da explicação.

A lista de mensagens segue uma ordem fixa — **SystemMessage** de confiança, **UserMessage** com dados estruturados da base de dados, historial alternado de **AiMessage/UserMessage** e, por fim, o input atual do utilizador isolado na sua própria **UserMessage** de origem não confiada (Figura C.4). A fronteira de *role* da API Mistral garante que este último nunca altera o comportamento definido no **SystemMessage**.

Os prompts de geração de tarefas (**TaskGenerationPrompt**) e de análise de dificuldade (**StruggleAnalysisPrompt**) não recebem qualquer input do utilizador final — são alimentados exclusivamente com dados estruturados do domínio, eliminando o vector de injeção nessas chamadas.

Camada 6 — Sanitização de Output (AiOutputSanitiser)

Todo o texto gerado pelo LLM passa pelo **AiOutputSanitiser** antes de ser persistido ou devolvido ao cliente, incluindo a resposta conversacional de explicação; o mecanismo e os pontos de aplicação estão descritos na Secção 10.1.

Resiliência e Observabilidade

A integração com a API Mistral é protegida por *circuit breaker*, *retry* com *backoff* exponencial e *timeout*, com *fail-safe* que repõe o *assignment* em **PENDING** caso a geração falhe irremediavelmente — garantindo que o estudante nunca fica bloqueado. A configuração exacta e as métricas instrumentadas (duração, falhas por tipo, estratégia de reutilização) estão detalhadas nos Apêndices C (Tabela C.9) e D (Tabela D.3).

O mecanismo de *Retrieval-Augmented Generation* que suporta a deduplicação semântica e a reutilização adaptativa de conteúdo referidas ao longo deste capítulo é detalhado no Capítulo 7.

Capítulo 7

Retrieval-Augmented Generation

Este capítulo dá continuidade à descrição do modelo de IA apresentada no Capítulo 6, detalhando o mecanismo de *Retrieval-Augmented Generation* (RAG) usado pelo Play4Change para deduplicação semântica e reutilização adaptativa de conteúdo. O ciclo de vida completo dos desafios entregues ao Learner é tratado em separado nos Capítulos 8 e 9, e a segurança aplicacional de ponta a ponta no Capítulo 10.

O Play4Change implementa um padrão RAG que difere do RAG documental clássico: em vez de recuperar fragmentos de texto para enriquecer um *prompt* a cada resposta, recupera vectores semânticos previamente computados para tomar decisões de reutilização e injeta contexto pedagógico estruturado nos prompts enviados ao LLM. O sistema mantém duas colecções vectoriais independentes em pgvector, executando pesquisa por similaridade cosseno em dois contextos distintos: deduplicação durante a geração de tarefas e branching adaptativo durante a sessão de dificuldade.

7.1 Armazenamento de Conhecimento

Base de Dados Vectorial

O armazenamento persistente de conhecimento assenta em PostgreSQL com a extensão **pgvector**, activada na migração V3. São mantidas duas colecções vectoriais independentes, cada uma com um índice IVFFLAT (predicado `WHERE embedding IS NOT NULL`, para que apenas registos com vector calculado entrem no plano de pesquisa): `task_templates.embedding vector(1024)`, que representa semanticamente o par título+descrição de cada tarefa gerada; e `struggle_events.embedding vector(1024)`, que representa o contexto de dificuldade de um aprendente (padrão de erro + descrição da tarefa + domínio), associado à tabela **adaptive_branches** que regista os ramos gerados para reutilização futura (migração V21).

O operador de distância cosseno do pgvector é `<=>`; a similaridade é derivada como `1 - (embedding <=> ?::vector)`, com valores no intervalo `[0, 1]`. O modelo de *embedding* é o `MistralAiEmbeddingModel`, que produz vectores de 1024 dimensões.

Pipeline de Ingestão

O conhecimento é ingerido em quatro fases executadas de forma assíncrona pelo `TaskGenerationOrchestrator` com o estado rastreado na coluna `current_phase` da tabela `topics`, seguindo a mesma máquina de estados já apresentada na Secção 4.2.2 do Capítulo 4.

Antes da fase `GENERATION`, o `ContentChunker` segmenta o texto extraído em janelas com sobreposição, seguindo o mesmo algoritmo de preservação de fronteiras de parágrafo já descrito na Secção 5.2 do Capítulo 5 (parâmetros na Tabela 6.1).

Por cada tarefa gerada pelo LLM num chunk, o pipeline calcula o embedding do par título+descrição e consulta o índice `IVFFLAT` antes de persistir: se a similaridade máxima face aos templates existentes no módulo for superior a 0.90, a tarefa é descartada como duplicado semântico. O diagrama de sequência completo deste pipeline, desde a submissão até à activação do tópico, foi já apresentado na Figura C.1 (Secção 4.2.2 do Capítulo 4).

7.2 Recuperação e Reinjeção de Contexto

O sistema executa recuperação vectorial em dois contextos distintos: geração de ramos adaptativos (via similaridade entre vectores de dificuldade) e geração de explicações (via joins relacionais, sem pesquisa vectorial).

Recuperação de Ramo Adaptativo

Quando um aprendente falha uma tarefa, o `HandleStruggleService` lança assincronamente a geração de um ramo adaptativo. O vector de entrada é construído concatenando `errorPattern`, `taskDescription` e `subjectDomain` (truncado a 8 000 chars), e embebido em 1024 dimensões. A pesquisa vectorial no `PgVectorDeduplicationService.findSimilarStruggle()` ordena os ramos já `COMPLETED` por distância cosseno ao vector de entrada, excluindo os ramos que o aprendente já viu, e devolve o mais próximo.

A estratégia de reutilização é determinada por dois limiares configuráveis em `application.yml`:

Tabela 7.1: Estratégias de reutilização adaptativa por limiar de similaridade

Similaridade	Estratégia	Comportamento
> 0.90	FULL_REUSE	Devolver ramo existente sem chamada ao LLM
[0.65, 0.90]	PARTIAL_REUSE	Reutilizar $\lfloor n \times \text{normSim} \rfloor$ tarefas; gerar o complemento via Mistral; persistir novo ramo
< 0.65	FRESH_GENERATION	Gerar ramo completo via Mistral; persistir embedding para reutilização futura

Para `PARTIAL_REUSE`, a proporção de reutilização é calculada por interpolação linear, $n_{\text{reuse}} = \lfloor n_{\text{total}} \times \frac{\text{sim}-0.65}{0.90-0.65} \rfloor$ — a mesma fórmula e o mesmo mecanismo são também usados na síntese de ramos adaptativos a partir de sessões de dificuldade (Secção 6.1 do Capítulo 6).

Assemblagem e Reinjeção de Contexto para Explicações

Quando o aprendiz esgota o nível máximo de dificuldade (`MAX_STRUGGLE_DEPTH = 1`), é criada uma sessão de explicação. O `ExplanationService.buildContext()` recupera e assembla contexto exclusivamente por joins relacionais — sem pesquisa vectorial — construindo o `ExplanationContext` a partir da descrição da tarefa, do objetivo do módulo e do domínio (truncados conforme a Tabela 6.1), do nível de audiência e língua do tópico, do padrão de erro traduzido, e do historial completo de sessões de dificuldade anteriores (profundidade, padrão de erro e títulos das tarefas adaptativas).

Este contexto é injetado no *prompt* de utilizador de `ExplanationPrompt.userExplanation()`, que inclui o historial completo de sessões de dificuldade do aprendiz nessa tarefa. Para as respostas conversacionais, a lista tipada de `ChatMessage` (descrita na Secção 6.1.3 do Capítulo 6) isola dados de controlo de conteúdo do utilizador; o ciclo completo de explicação está ilustrado na Figura C.5 (Secção 9.3 do Capítulo 9, diagrama no Apêndice C).

7.3 Edição de Conteúdo e Invalidação

A base de conhecimento vectorial pode tornar-se incoerente quando o conteúdo pedagógico é alterado por um administrador. O sistema trata este problema de duas formas, consoante o alcance da edição.

Na **edição pontual** de uma tarefa (`AdminTaskService.updateTask()`), o *template* é atualizado e as suas instâncias recriadas, mas o *embedding* da versão anterior é mantido — uma inconsistência deliberadamente aceite, já que só afeta a deduplicação em gerações futuras, não a entrega ao aprendiz.

Na **regeneração completa** de um tópico (`POST /admin/topics/{id}/regenerate`, rejeitada se já estiver em `GENERATING`), o módulo antigo é marcado como superado e depois eliminado, acionando uma cascata que purga `task_templates`, `task_instances`, `struggle_events` e `adaptive_branches` do índice IVFFLAT — todos os vetores são recalculados de raiz.

A diferença central é, portanto, o tratamento do *embedding*: aceite desatualizado na edição pontual, recalculado por completo na regeneração.

O ciclo de vida completo dos desafios entregues ao Learner — geração (Capítulo 8), e atribuição, remediação adaptativa (*struggle path*) e tutoria com IA (Capítulo 9) — é tratado em separado nesses capítulos.

Capítulo 8

Geração de Desafios

Este capítulo dá continuidade à descrição do modelo de IA e do RAG apresentada nos Capítulos 6 e 7, detalhando a geração de desafios (*tasks*) por Inteligência Artificial no Play4Change — do modelo de dados que os representa ao *pipeline* assíncrono de quatro fases que os produz a partir do conteúdo ingerido. A atribuição destes desafios aos utilizadores, a remediação adaptativa e a tutoria com IA são tratadas em separado no Capítulo 9.

8.1 Modelo de Dados

Cada desafio existe em dois níveis de abstracção complementares:

- **TaskTemplate** — definição canónica de um desafio, gerada pelo LLM, com a resposta correta sempre fixada no índice 0 e um *embedding* de 1024 dimensões para deduplicação.
- **TaskInstance** — variante com distratores alternativos: partilha a resposta correta do *template* mas oferece opções diferentes, até `instancesPerTask = 5` por *template*, permitindo que utilizadores distintos vejam a mesma questão com opções erradas variadas.

Esquema de campos completo no Apêndice C (Tabela C.3).

O facto de o `correctAnswer` ser sempre o índice 0 no *template* e na instância é uma invariante da geração: o *system prompt* do `TaskGenerationPrompt` instrui explicitamente o LLM a colocar a resposta correta sempre na primeira posição, o que simplifica a lógica de validação e elimina ambiguidade na persistência.

8.2 Pipeline de Geração em Quatro Fases

A geração é um processo *batch* assíncrono, orquestrado pelo `TaskGenerationOrchestrator`, com estado rastreado na coluna `current_phase` pela mesma máquina de estados da Secção 4.2.2 do Capítulo 4.

A fase **ANALYSIS** marca todos os templates existentes como `is_current = FALSE` e cria o novo módulo. Na fase **GENERATION**, por cada *chunk* (Tabela 6.1), o `TaskGenerationPrompt`

instrui o LLM a gerar tarefas com quatro opções, resposta correta no índice 0, sem sobreposição semântica com tarefas já geradas — deduplicação por pgvector já detalhada na Secção 7.1 do Capítulo 7. Na fase `INDEXING`, o `BatchInstanceGenerationService` processa os templates em lotes de 10 e gera até 5 instâncias por template via chamadas LLM adicionais.

O mecanismo de *retry* com *schema reminder* (Secção 6.1 do Capítulo 6) aplica-se a cada *chunk* nesta fase. A falha no primeiro *chunk* aborta a geração (`FAILED`); falhas em *chunks* subsequentes são toleradas, preservando as tarefas já persistidas. As seis garantias de qualidade aplicadas a cada questão gerada estão detalhadas no Apêndice D (Tabela D.1).

Capítulo 9

Atribuição, Remediação e Tutoria com IA

Este capítulo dá continuidade à descrição da geração de desafios apresentada no Capítulo 8, descrevendo o percurso do Learner desde a atribuição diária de um desafio até à sua resolução — passando pelo percurso adaptativo de remediação (*struggle path*) e pelo modo de tutoria conversacional com IA quando a remediação não é suficiente. Na nomenclatura interna do sistema, os desafios são denominados *tasks* e cada conceito de negócio tem correspondência direta a entidades de domínio bem delimitadas. A segurança aplicacional de ponta a ponta é tratada em separado no Capítulo 10.

9.1 Atribuição aos Utilizadores

Entidades de Atribuição

Dois agregados gerem o estado da aprendizagem por utilizador:

- **Enrollment** — inscrição ativa de um utilizador num módulo, com o dia corrente do percurso, o estado (`ACTIVE` | `COMPLETED` | `EXPIRED` | `PAUSED`) e os pontos e *streak* acumulados.
- **TaskAssignment** — atribuição concreta de um desafio a um utilizador num dia, com a permutação de opções aplicada, o prazo de 24 horas e o estado de submissão.

Esquema de campos completo no Apêndice C (Tabela C.4).

Fluxo Pull-based

A atribuição de desafios é *pull-based*: o utilizador solicita a tarefa do dia via `GET /tasks/today`, e o serviço resolve quatro casos mutuamente exclusivos, por ordem de precedência: (1) sessão de *struggle* aberta; (2) sessão de explicação ativa; (3) assignment já existente para o `dayIndex` atual; (4) criação de novo assignment.

No caso (4), o sistema selecciona aleatoriamente uma das **TaskInstance** disponíveis (variedade de distratores), gera uma permutação aleatória das quatro opções (**optionOrder**), e define **dueAt** = **now()** + 24h.

Permutação das Opções e Penalizações por Atraso

A **optionOrder** é uma lista de índices (ex.: [2,0,3,1]) que define a ordem de apresentação no ecrã. A resposta correta, sempre no índice 0 da instância, aparece numa posição aleatória para o utilizador. A resposta submetida é um índice de posição de ecrã; o sistema resolve o índice canónico antes de validar, via **optionOrder[selectedOption]**.

A submissão após **dueAt** é aceite mas penalizada em função do atraso *face ao próprio dueAt* (e não do tempo total desde a atribuição): 100% dos pontos sem atraso, 75% até 24 h, 50% até 72 h, e 25% (mínimo 1 ponto) além disso.

9.2 Consolidation Path (Struggle Path)

Quando um utilizador responde incorrectamente, o sistema não termina a interação — desencadeia automaticamente uma sessão de remediação adaptativa (*struggle session*). O objetivo é diagnosticar a causa do erro e apresentar três sub-tarefas *scaffolded* que devolvam o utilizador ao nível da tarefa original.

Classificação do Padrão de Erro

Antes de invocar a IA, o **ErrorPatternClassifier** classifica a falha de forma determinística em três categorias, com base no contexto da submissão:

Tabela 9.1: Padrões de erro classificados pelo **ErrorPatternClassifier**

Padrão	Condição	Estratégia de remediação
TIME_PRESSURE	Submissão após dueAt	Tarefas panceadas, passo a passo
READING_ERROR	Opção seleccionada adjacente à correta ($ pos_sel - pos_corr == 1$)	Instruções mais explícitas e claras
WRONG_CONCEPT	Qualquer outro caso	Re-explicação do conceito de raiz

Geração Assíncrona da Sessão

Após classificar o erro, o **TaskService** persiste o resultado e delega ao **HandleStruggleService** via **triggerAsync** — a resposta HTTP é devolvida imediatamente ao utilizador, sem esperar pela geração do ramo adaptativo pelo LLM. Em caso de falha ou *timeout* do LLM, a sessão é marcada como **ABANDONED** e o assignment original é reposto em **PENDING** — o utilizador nunca fica bloqueado.

Submissão e Resolução das Tarefas Adaptativas

O utilizador acede às três sub-tarefas via **GET /struggle/enrollment/{enrollmentId}**. Cada submissão pode resultar em quatro desfechos, avaliados após a conclusão de todas as tarefas:

(1) **ainda não concluídas** — devolver resultado parcial; (2) **todas corretas** — marcar sessão como **RESOLVED** e repor assignment em **PENDING**; (3) **falha**, **depth** < **MAX_STRUGGLE_DEPTH** — gerar nova sessão de *struggle* encadeada; (4) **falha**, **depth** ≥ **MAX_STRUGGLE_DEPTH** (= 1) — criar sessão de explicação por IA.

A profundidade máxima de remediação está fixada em **MAX_STRUGGLE_DEPTH** = 1 — uma decisão de desenho deliberada, assente em dois argumentos. O primeiro é pedagógico: as sub-tarefas partilham o formato de escolha múltipla do desafio original, pelo que encadear níveis adicionais do mesmo formato tem retorno decrescente — uma incompreensão que sobrevive a três sub-tarefas *scaffolded* dificilmente se resolve com mais três no mesmo molde, e a tutoria conversacional (Secção 9.3) é o mecanismo desenhado precisamente para esse caso. O segundo é operacional: cada nível adicional multiplica as chamadas à API de IA e prolonga o tempo até o Learner regressar ao fluxo principal. O compromisso é reconhecido: tópicos avançados não dispõem de patamares intermédios antes do escalamento, e tornar a profundidade configurável por tópico fica identificado como trabalho futuro (Secção 13.5).

As tarefas adaptativas atribuem zero pontos e não afetam o *streak* — são modo de aprendizagem puro.

9.3 Tutoria com IA

Quando o utilizador esgota toda a profundidade de remediação sem sucesso, o sistema ativa o modo de tutoria conversacional com IA. Esta sessão sintetiza o historial completo de erros e oferece uma explicação holística em prosa, seguida de um diálogo de esclarecimento sem limite de turnos.

Estados da Sessão de Explicação

A sessão de explicação percorre três estados:

GENERATING → ACTIVE → RESOLVED

A transição **GENERATING** → **ACTIVE** ocorre quando a geração pelo LLM termina com sucesso. Se a geração falhar ou esgotar o *timeout* (60s via `withTimeoutOrNull`), a sessão transita diretamente para **RESOLVED** e o assignment é repostado em **PENDING** (*fail-safe* — o utilizador nunca fica bloqueado). A transição para **RESOLVED** pelo utilizador, via `POST /explanation/{sessionId}/resolve`, também repõe o assignment em **PENDING**, devolvendo-o ao fluxo diário normal.

Geração Holística e Diálogo Conversacional

A geração holística é assíncrona (`generationExecutor`), permitindo que o utilizador navegue para o ecrã de explicação enquanto a geração ocorre em *background*, com *polling* até o

estado ser **ACTIVE**. O contexto injetado inclui o historial completo de sessões de dificuldade, assemblado por joins relacionais sem pesquisa vectorial (Secção 7.2 do Capítulo 7).

Para o diálogo conversacional, cada mensagem é processada com o historial completo, usando a mesma estrutura multi-mensagem tipada descrita na Secção 6.1.3 do Capítulo 6 para isolar o input do utilizador. A Figura C.5 (Apêndice C) detalha o fluxo completo desde a criação assíncrona até à resolução.

Capítulo 10

Segurança da Aplicação

Este capítulo encerra a descrição técnica do sistema iniciada nos Capítulos 4, 5, 6, 7, 8 e 9, detalhando as decisões de segurança tomadas em resposta a vetores de ataque comuns e conhecidos.

A segurança não foi o foco central deste projeto, mas foram tomadas ao longo do desenvolvimento um conjunto de decisões concretas em resposta a vetores de ataque comuns e conhecidos, com controlos em várias camadas da arquitetura: proxy reverso (Nginx), servidor de aplicação (Spring Boot), agente de IA (LangChain4j) e cliente web (React/Vite). A lista dos vetores considerados e do controlo correspondente está resumida na Secção 10.5, evitando duplicar aqui a mesma informação.

10.1 Proteção Contra XSS

A proteção contra *Cross-Site Scripting* combina três técnicas complementares e independentes.

Sanitização de Output de IA

O `AiOutputSanitiser` sanitiza todo o conteúdo gerado pelo modelo Mistral antes de ser persistido, usando `Jsoup.parse(input).text()` para remover elementos HTML, decodificar entidades e descartar o conteúdo de `<script>/<style>`. É aplicado a cada campo `String` do `TaskTemplate` gerado e às respostas conversacionais de explicação — mitigando o vetor de páginas raspadas (*scraped*) que possam conter HTML malicioso e ser reproduzidas literalmente pelo modelo.

Content Security Policy via Nginx

O Nginx aplica uma *Content Security Policy* restritiva às respostas da SPA estática (bloco `location /` do `nginx.conf`) — `default-src` e `script-src` limitados a `'self'` (sem `unsafe-eval` nem CDNs externos), `style-src` com `'unsafe-inline'` apenas pelo necessário aos estilos gerados pelo Vite/React no *build*, e `frame-ancestors 'none'` contra *clickjacking*. Ao contrário dos restantes *headers* de segurança (Tabela C.8, Secção 10.4), que são globais a todas as

respostas, a CSP é — por decisão documentada (ADR-018) — aplicada apenas a este bloco, não às respostas JSON da API proxied em `/play4change-server/`, `/api`, `/swagger-ui` ou `/v3`.

React JSX Auto-escaping

O React aplica *escaping* automático de todas as expressões `{expression}` ao renderizar no DOM. Mesmo que a base de dados contivesse HTML residual (falha nas camadas anteriores), o React converteria `<` e `>` em entidades HTML antes da inserção no DOM, eliminando XSS reflexo em componentes bem formados.

10.2 Rate Limiting por IP

O *rate limiting* é implementado no filtro `RateLimitFilter`, com ordem `HIGHEST_PRECEDENCE` na cadeia de filtros Spring, garantindo que a verificação ocorre antes de qualquer autenticação ou lógica de negócio. O algoritmo utilizado é *token bucket* via `Bucket4j`, com janela de *refill* de 10 minutos. Os *buckets* são mantidos em cache `Caffeine` com TTL de 15 minutos e capacidade máxima de 50 000 entradas. Cada *bucket* é identificado pela chave composta `<ip>:<path_normalizado>`, isolando os limites por recurso e por origem.

O `IpExtractor` extrai o IP do header `X-Forwarded-For` definido pelo Nginx, mas rejeita endereços privados para prevenir *spoofing* por injeção de header, caindo nesse caso para `remoteAddr`. Os limites concretos por *endpoint* são apresentados no Apêndice D (Tabela D.2); documenta-se aí também um problema entretanto corrigido, em que quatro regras estavam configuradas contra padrões de rota desalinhados — duas delas resíduo da funcionalidade de *Peer Review* entretanto removida, e as outras duas (submissão de tarefa e de sub-tarefa de *struggle*) efetivamente sem limite de taxa até à correção.

10.3 Validação e Sanitização de Inputs

A validação de *inputs* é realizada em quatro níveis independentes: **Cliente** (TypeScript, otimização de UX — não controlo de segurança — para email, PDF e URLs); **Servidor** (Jakarta Bean Validation nos DTOs de entrada, `@NotBlank`/`@Email`/`@Size`, devolvendo 400 sem chegar à camada de serviço); **Prevenção de SSRF** (`UrlSsrValidator`: esquema HTTPS obrigatório, resolução DNS e rejeição de IPs privados/*loopback*/*link-local*, detalhado na Secção 5.1 do Capítulo 5); e **Limites de Contexto para IA** (`AiContextLimits`, Tabela 6.1, Secção 6.1 do Capítulo 6), aplicados antes de construir os prompts independentemente do tamanho do *input* original.

10.4 Outros Controlos de Segurança

Autenticação JWT com Validação de Arranque

O `JwtSecretStartupGuard` valida o segredo JWT no arranque, falhando se tiver menos de 64 caracteres ou for igual ao valor de desenvolvimento por omissão (apenas em perfil `prod`, ver Secção 4.3.3); o `CredentialStartupGuard` verifica analogamente as credenciais de serviços externos. Estes guardas impedem a promoção de configurações de desenvolvimento para produção.

Rotação de Refresh Token com Detecção de Reutilização

O mecanismo de rotação com família (Secção 4.3) deteta roubo de token: se um token já marcado como `used = true` for apresentado, toda a família é revogada e um evento `SECURITY` é emitido.

RBAC e CORS Restritivo

O `SecurityConfig` define rotas protegidas por papel (`/admin/**` exige `ROLE_ADMIN`; Swagger restrito a admin em produção), validado no `JwtAuthFilter` contra a lista `VALID_ROLES`. O `WebMvcConfig` configura CORS com lista de origens explícita, verificada ao arranque (rejeita *wildcards* e entradas em branco), definidas em produção via `APP_CORS_ALLOWED_ORIGINS`.

Headers HTTP de Segurança (Nginx)

O Nginx acrescenta cinco *headers* de segurança no total — `Strict-Transport-Security`, `X-Content-Type-Options`, `X-Frame-Options` e `Referrer-Policy`, aplicados globalmente a todas as respostas, e a *Content-Security-Policy* já descrita na Secção 10.1, aplicada apenas ao bloco da SPA estática — cobrindo *downgrade* para HTTP, MIME-sniffing, *clickjacking* e vazamento de *referrer*; a configuração exacta de cada um está listada no Apêndice C (Tabela C.8).

Segurança do Scraper

O scraper Playwright opera com *timeout* de navegação de 25 s, *timeout* total de 35 s, e remoção ativa de `<script>`, `<style>`, `<iframe>`, `<nav>`, `<footer>` antes de extrair texto — em complemento ao `UrlSsrValidator` que valida a URL antes de qualquer pedido ao scraper.

10.5 Vetores de Ataque Considerados e Mitigações

Não foi conduzida uma auditoria de segurança formal nem uma análise de risco quantificada; a segurança foi endereçada de forma pragmática, à medida que cada vetor de ataque relevante para a arquitetura concreta do sistema foi identificado. A Tabela 10.1 resume os dez vetores considerados e a decisão de mitigação tomada para cada um.

Tabela 10.1: Vetores de ataque considerados e controlo implementado

Vetor	Controlo implementado	Observação
XSS via conteúdo gerado por IA	<code>AiOutputSanitiser</code> + CSP (parcial, §10.1) + <i>auto-escaping</i> do React	—
Prompt Injection	Multi-mensagem tipada + <code>AiContextLimits</code>	Mitiga ataques simples; o Mistral não garante resistência total a <i>inputs</i> adversariais sofisticados
Custo descontrolado de chamadas à API de IA	Rate limiting por IP (10 req/10 min em <code>/explanation</code> , entre outros)	Mitiga abuso de um único atacante, não um ataque distribuído a partir de múltiplos IPs
Roubo de <i>refresh token</i>	Rotação com família + revogação total em caso de reutilização	Janela de 7 dias de exposição se o token for exfiltrado; impacto limitado pela TTL de 15 min do <i>access token</i>
Segredo JWT por omissão em produção	<code>JwtSecretStartupGuard</code> impede o arranque	—
SSRF via URL de ingestão	<code>UrlSsrValidator</code>	—
Força bruta em <i>magic-link</i>	Rate limiting (5 req/10 min)	—
<i>Spoofing</i> de IP no rate limiting	<code>IpExtractor</code> rejeita endereços privados no header	—
Escalada de privilégios (Learner → Admin)	RBAC + validação de papel no JWT	—
CORS <i>wildcard</i>	Verificação de lista de origens ao arranque	—

Três destas decisões merecem uma nota adicional, por serem as que deixam uma margem de exposição residual mais visível: a proteção contra *prompt injection* não é absoluta (depende do comportamento do modelo, não apenas da separação de mensagens); o *rate limiting* por IP não impede um ataque distribuído a partir de múltiplos endereços; e a janela de 7 dias do *refresh token* só é neutralizada se o XSS acima estiver de facto bem mitigado. Nenhuma destas foi objeto de um exercício de mitigação adicional — ficam identificadas como pontos a reforçar caso o sistema avance para produção com utilização real.

Capítulo 11

Suite de Testes Automatizados

A validação automatizada do Play4Change assenta em três camadas complementares: testes unitários que verificam o comportamento isolado do domínio e dos serviços de aplicação, testes de integração que exercitam a camada HTTP em contexto Spring parcial, e testes de segurança que confrontam as defesas com os vectores de ataque considerados (§10.5). Juntas, estas camadas cobrem 50 ficheiros de teste distribuídos pelos módulos *server* e *play4change-web*. A avaliação de usabilidade com doze utilizadores reais é tratada em separado no Capítulo 12.

11.1 Suite Automatizada e Estratégia de Testes

A estratégia assenta em três princípios: **isolamento total pelo padrão hexagonal** (todas as dependências externas são interfaces, substituídas por duplicados *Mockk*, pelo que os testes de domínio correm em memória pura); **Either<Error, T> como contrato de resultado** (a biblioteca Arrow obriga cada caso de uso a devolver `Either<DomainError, T>` em vez de lançar exceções, tornando os casos de insucesso tão verificáveis como os de sucesso); e **separação em três níveis** — unitário, integração (`@WebMvcTest`) e segurança — resumidos, com a dimensão da suite por módulo, na Tabela 11.1.

Tabela 11.1: Suite de testes automatizados: dimensão e caracterização por nível.

Nível / Módulo	Ficheiros	Ferramenta	Contexto Spring	Deps. externas
Unitário (server)	26	JUnit 5 + Mockk	Nenhum	Todas em mock
Integração (server)	11	@WebMvcTest + MockMvc	Parcial (camada web)	Serviços em mock
Frontend (React/Vite)	13	Vitest + RTL	—	Mock
Total	50			

Os testes **unitários** cobrem o domínio e a aplicação em memória pura: autenticação (revogação de família de tokens JWT em caso de roubo, ameaça R04 §10.5; consumo atómico do *magic link*), entrega de desafios (baralhagem determinística de opções por semente de utilizador §9.1; guarda de arranque que impede `devMode` em produção) e deteção de dificuldades (ciclos no grafo de pré-requisitos por DFS; classificação de padrões de erro por prioridade estrita).

Os testes de **integração** exercitam a camada HTTP completa via `@WebMvcTest` — serialização JSON, filtros de segurança, validação de *beans* — em dez controladores REST (autenticação, pré-requisitos, *struggle*, relatórios, preferências, crachás, estatísticas, Swagger) e num teste de *rate limiting* particularmente rigoroso. O `RateLimitTest` substitui o `TimeMeter` real por um `ManualTimeMeter`, tornando deterministas quatro casos (limite estrito, reset de janela, isolamento por IP, cabeçalho `Retry-After`); o complementar `RateLimitSubmissionPathsTest` cobre a correção do desalinhamento de rota nos *endpoints* de submissão (§10.2 do Capítulo 10).

Os testes de **segurança** validam as defesas identificadas na análise de risco (§10.5): SSRF (`UrlSsrValidatorTest`, 10 casos cobrindo *loopback*, redes privadas, *link-local* e esquema não-HTTPS — detalhe de casos na Tabela C.7, Apêndice C), sanitização de *output* da IA (`AiOutputSanitiserTest`, 5 casos), força bruta no *magic link* (coberta pelo `RateLimitTest` acima) e dois guardas de arranque anti-fraude (`devMode` e `TaskDeliveryStartupGuard`) que impedem cadências de desenvolvimento de chegarem a produção.

Capítulo 12

Testes de Usabilidade

Este capítulo dá continuidade à validação automatizada apresentada no Capítulo 11, descrevendo a avaliação de usabilidade conduzida com doze utilizadores reais através da escala SUS (*System Usability Scale*).

Antes de apresentar os resultados, importa delimitar o que esta avaliação mede: a escala SUS quantifica a perceção subjetiva de facilidade de utilização e de aprendizagem da interface — não a aquisição efetiva de competências digitais pelo utilizador. Uma pontuação SUS elevada indica que a aplicação é fácil de usar; não constitui, por si só, evidência de que os Learners melhoraram as suas competências no referencial DigComp 2.2. Este relatório não contém dados (avaliação de competências antes/depois da utilização, ou medição longitudinal de desempenho) que permitam sustentar essa inferência.

12.1 Metodologia

A avaliação de usabilidade foi conduzida através de um questionário Google Forms aplicado a 12 participantes entre 8 e 12 de Junho de 2026 (4 estudantes actuais do ISEL, 7 testadores externos, 1 antigo aluno), organizado em quatro secções: avaliação global (escala 1–5), quatro declarações de envolvimento (Likert 1–5), a escala SUS (*System Usability Scale*) de Brooke [36] (10 itens, Likert 1–5), e questões abertas.

12.2 Análise SUS

A pontuação SUS segue a fórmula de Brooke: subtrai-se 1 aos itens ímpares (positivos) e o valor a 5 nos pares (negativos), soma-se os 10 resultados e multiplica-se por 2,5, numa escala de 0 a 100. Este valor de 0 a 100 **não é uma percentagem**: é um *score* composto que só é interpretável por comparação com um referencial externo [37]. Segue-se por isso, para interpretar os resultados, a orientação do Nielsen Norman Group sobre a análise do SUS [37], que reporta o *benchmarking* de Sauro a mais de 500 estudos: a pontuação SUS média observada na indústria é 68,0, e um sistema com pontuação ≥ 80 posiciona-se no top

10% dos sistemas avaliados nesse universo de estudos. A Tabela 12.1 apresenta as pontuações obtidas face a este referencial.

Tabela 12.1: Pontuações SUS por participante ($n = 12$), face à média de referência da indústria (68,0) [37].

Part.	Perfil	Face à referência (68,0)	SUS
P01	Estudante	Acima (top 10%*)	80,0
P02	Estudante	Acima (top 10%*)	97,5
P03	Externo	Acima (top 10%*)	90,0
P04	Externo	Abaixo	45,0
P05	Alumni	Abaixo	35,0
P06	Externo	Abaixo	50,0
P07	Externo	Acima	77,5
P08	Externo	Acima (top 10%*)	95,0
P09	Estudante	Abaixo	60,0
P10	Externo	Acima (top 10%*)	97,5
P11	Externo	Acima (top 10%*)	92,5
P12	Estudante	Acima (top 10%*)	92,5
Média			76,0
Mediana			85,0

*Pontuação ≥ 80 , o limiar que Sauro reporta como correspondente ao top 10% dos estudos de referência [37].

A média de 76,0 está 8,0 pontos acima da média de referência de 68,0 reportada por Sauro para mais de 500 estudos [37], e sete dos doze participantes (58,3%) atingiram uma pontuação igual ou superior a 80, o limiar que posiciona um sistema no top 10% desse universo de referência. A discrepância entre média (76,0) e mediana (85,0) reflecte dois *outliers* (P04 = 45, P05 = 35) que puxam a média para baixo — sem eles, a média sobe para 83,25. A avaliação global obteve uma média de 4,25 em 5.

Uma ressalva de leitura aplica-se a estes números: comparar doze respostas contra um *benchmarking* assente em centenas de estudos é indicativo, não uma inferência estatisticamente robusta.

As declarações de envolvimento (Tabela 12.2) mostram o design visual como a mais bem pontuada (4,33) e o encorajamento a regressar diariamente como a mais baixa (3,92), sugerindo margem de melhoria na retenção.

12.3 Análise Qualitativa

As respostas abertas revelaram pontos fortes e oportunidades de melhoria consistentes.

Pontos fortes. A explicação adaptativa após resposta errada foi apontada como melhor funcionalidade por quatro participantes; a intuitividade da interface por outros quatro; e a adaptabilidade da IA ao desempenho individual por três. Os desafios diários e a relevância das questões foram igualmente referidos positivamente.

Tabela 12.2: Médias das declarações de envolvimento (escala Likert 1–5, $n = 12$).

Declaração	Média
A aplicação motivou-me a aprender sobre educação cívica / cidades inteligentes	3,83
Os desafios pareceram relevantes para problemas do mundo real	4,00
O design visual e a estética do jogo foram apelativos	4,33
Senti-me encorajado/a a regressar e completar desafios diariamente	3,92
Os caminhos de aprendizagem IA adaptaram-se eficazmente ao meu desempenho	4,08

Problemas técnicos. A mais grave foi uma *race condition* visual no caminho de consolidação: a resposta era marcada correta e, meio segundo depois, aparecia como errada. Foram também reportados a barra de pesquisa oculta durante a inserção de e-mail, respostas corretas marcadas intermitentemente como erradas, e questões repetidas em caminhos alternativos. Os dois participantes com pontuações mais baixas (P04, P05) apontaram ainda a confusão causada pela transição abrupta para um novo desafio após resposta errada, sem explicação — comportamento que ocorre antes de a sessão de *struggle* ser criada.

Perfis de afinidade tecnológica. A observação direta revelou uma segunda linha de segmentação, além do vínculo institucional (Tabela 12.1): P04 e P05, os dois *outliers* negativos, pertencem a um perfil de utilização tecnológica mínima, sugerindo que a sua pontuação reflecte não só o bug de transição identificado acima, mas também uma dificuldade de base com aprendizagem digital autónoma — precisamente o perfil que mais beneficiaria do Play4Change (retomada em §13.5).

Sugestões de melhoria. As mais recorrentes foram funcionalidades sociais (amigos, tabelas de classificação), diversificação de tópicos, maior concisão das explicações IA em mobile, e personalização geográfica das questões por localização.

Conclusão da avaliação. Os resultados SUS indicam uma usabilidade global consistentemente acima da média de referência da indústria (68,0), com a maioria dos participantes (sete em doze) a atingir o limiar de top 10% (≥ 80) reportado por Sauro/NNGroup [37]. Os dois *outliers* convergem num problema de comunicação de estado após erro, mitigável com uma transição animada, mas reflectem também uma barreira de entrada mais alta para o perfil de menor afinidade tecnológica.

Capítulo 13

Discussão

Este capítulo confronta os resultados com os objetivos propostos, identifica o que correu melhor e pior, posiciona o Play4Change face ao estado da arte, e aponta direções de trabalho futuro.

13.1 Objetivos Atingidos

Os cinco objetivos operacionais foram todos atingidos, com diferentes graus de exigência.

OBJ-1 — Plataforma adaptativa DigComp 2.2

O sistema implementa inscrição com gating de pré-requisitos, entrega calendarizada de tarefas, remediação por dificuldade e escalamento para tutoria IA. O alinhamento com o DigComp 2.2 depende da curação humana dos tópicos, não de verificação automática. Na avaliação de usabilidade (Capítulo 12), sete dos doze participantes (58,3%) atingiram uma pontuação SUS de 80 ou superior — o limiar que Sauro reporta como correspondente ao top 10% de mais de 500 estudos de referência [37].

OBJ-2 — Pipeline de geração por IA

O pipeline — *ingestion*, *chunking*, geração com *schema reminder*, deduplicação semântica — funciona de ponta a ponta e degrada graciosamente em caso de falha (circuit breaker, retry). As seis garantias de qualidade estão detalhadas no Apêndice D.

OBJ-3 — Remediação adaptativa

O `ErrorPatternClassifier` classifica cada erro em três padrões e a reutilização semântica de ramos (§7.2) reduz custos de API. A limitação principal é a profundidade fixa (`MAX_STRUGGLE_DEPTH = 1`), analisada criticamente em §13.3.

OBJ-4 — Tutoria com IA

A `ExplanationSession` (§9.3) cobre geração assíncrona, diálogo multi-turno e retorno ao fluxo principal. Foi a funcionalidade mais valorizada na avaliação qualitativa (§12.3).

OBJ-5 — Segurança e qualidade

A segurança não foi o foco do projeto, mas dez vetores de ataque concretos foram endereçados (§10.5), três com exposição residual reconhecida (nível Médio); a suite de segurança valida SSRF, XSS, magic link e controlo de acesso ao Swagger. Não há pretensão de cobertura exaustiva.

13.2 O que Correu Bem

Arquitetura hexagonal

Modelar todas as dependências externas como interfaces (§4.1) foi a decisão mais rentável do projeto: torna a suite de testes rápida e determinística (§11.1), a ponto de permitir substituir o próprio relógio por um `ManualTimeMeter` (§11.1) e tornar 100% deterministas até os testes de janelas temporais de *rate limiting*.

Reutilização semântica de ramos

A reutilização por similaridade coseno (§7.2, Tabela 7.1) reduz o custo de API à medida que a base de ramos cresce, sem intervenção humana — o contributo técnico mais original do sistema.

Resiliência do pipeline de IA

Circuit breaker, degradação graciosa e *fail-safe* tornam o sistema tolerante a falhas intermitentes do Mistral, com visibilidade via o alerta `AiCircuitBreakerOpen`.

Autenticação sem senha

O fluxo *magic link* elimina a gestão de passwords e cobre as ameaças relevantes ao modelo de autenticação (§4.3).

13.3 Limitações e Dificuldades

Conformidade de schema do LLM

O Mistral não garante JSON válido em todas as chamadas. O *schema reminder* mitiga, não elimina, a falha; sem dados de produção não é possível quantificar a taxa real de ocorrência.

Profundidade fixa de struggle

A escolha de `MAX_STRUGGLE_DEPTH = 1`, justificada na Secção 9.2, deixa tópicos avançados sem patamares intermédios antes do escalamento para tutoria — o compromisso identificado fica por endereçar em trabalho futuro (§13.5).

Falta de dados de produção

O sistema instrumenta 20 métricas e oito alertas, mas a validação assentou em 12

participantes num ambiente controlado. Latências reais, distribuição de estratégias de reutilização e taxa de deduplicação sob carga permanecem por medir.

Fornecedor de IA único

O sistema está acoplado ao Mistral; a abstração LangChain4j isola a lógica de negócio, mas os prompts e os *embeddings* estão calibrados a este modelo — migrar exigiria recalibração e *re-embedding* de todo o catálogo.

Contexto *Report* por integrar

O mecanismo de sinalização de tarefas incorretas (contexto *Report*, Secção 4.1) está implementado no servidor e no painel web, mas não ligado a nenhum cliente — restam por decidir a política de suspensão de conteúdo, as salvaguardas contra falsos alarmes, e o tratamento de instâncias já entregues.

13.4 Comparação com o Estado da Arte

O Play4Change cruza três linhas do Capítulo 2: plataformas gamificadas, sistemas de tutoria inteligente (ITS) e geração de conteúdo por LLM.

Face às plataformas gamificadas

(Tabela 2.1, §2.6) Nenhuma gera conteúdo próprio a partir de fontes documentais de uma instituição, mas todas oferecem hoje mais funcionalidades sociais e de retenção — a lacuna mais relevante para o trabalho futuro.

Face aos ITS clássicos

(Cognitive Tutor, ALEKS) Estes usam modelos de utilizador explícitos (BKT, PFA), mais precisos diagnosticamente; o Play4Change usa uma heurística mais simples — três padrões de erro — trocando precisão por generalidade: funciona em qualquer domínio sem dados de calibração histórica.

Face ao uso direto de LLM para tutoria

(ChatGPT, Claude) É flexível mas sem percurso definido, gamificação ou isolamento de input. O Play4Change estrutura a progressão (tarefa → struggle → explicação → re-tentativa), gamifica, garante qualidade por schema e deduplicação, e isola o input do utilizador em mensagens tipadas.

13.5 Trabalho Futuro

Duas direções têm prioridade estratégica:

Funcionalidades sociais e de retenção

Amigos, tabelas de classificação, conquistas sobre a infraestrutura de `micro_competences`

já existente, e notificações comportamentais — respondem à sugestão mais recorrente da avaliação de usabilidade e à declaração de envolvimento mais baixa (regresso diário, 3,92/5).

Reconhecimento formal via Open Badges e créditos ECTS

Tornaria o sistema de crachás atual, hoje interno e não portátil, exportável e verificável externamente, com potencial para micro-créditos ECTS na rede U!REKA.

A Tabela 13.1 resume seis direções de menor prioridade, todas viáveis sobre a arquitetura atual sem alterações estruturais.

Tabela 13.1: Direções de trabalho futuro de prioridade secundária

Direção	Descrição
Explicações adaptadas ao móvel	Comprimento configurável por <code>audienceLevel</code> e canal, ou versão resumida expansível — as explicações em prosa são hoje longas para ecrãs pequenos.
Personalização geográfica	Injetar a localização do dispositivo no contexto de geração, adaptando questões ao município do utilizador.
Profundidade variável de struggle	<code>MAX_STRUGGLE_DEPTH</code> configurável por tópico, baseado em <code>wrongAttemptCount</code> e na distribuição de padrões de erro.
Análise de desempenho das questões	Painel sobre <code>tasks_submitted_total</code> que sinalize questões com taxa de acerto <20% ou >95%.
Validação automática de alinhamento DigComp 2.2	Segundo prompt que classifique a competência testada e sinalize incongruências com o módulo.
Suporte nativo multi-plataforma	Kotlin Multiplatform permite compilação nativa para iOS; hoje entrega-se apenas Android.

Duas direções estratégicas, não necessariamente exclusivas, ficam em aberto — aprofundar a vertente universitária para o público tecnologicamente confortável, ou investir em acompanhamento humano para o perfil de menor afinidade digital — retomadas na síntese final do Capítulo 14.

Capítulo 14

Conclusão

O Play4Change respondeu a uma lacuna concreta: a ausência de uma plataforma de aprendizagem adaptativa que integrasse nativamente o referencial DigComp 2.2 e gerasse o seu próprio conteúdo pedagógico, por IA, a partir de fontes documentais fornecidas por um formador. O resultado é uma plataforma multi-cliente completa — painel de administração web, aplicação móvel para Learners e um *backend* que orquestra ingestão de conteúdo, geração por IA, indexação vetorial e remediação adaptativa. Os cinco objetivos operacionais definidos no Capítulo 1 foram todos alcançados (Capítulo 13, §13.1), com uma avaliação de usabilidade a confirmar uma pontuação SUS média de 76,0 — 8 pontos acima da média de referência da indústria [37] — e a tutoria adaptativa por IA identificada como a funcionalidade mais valorizada — resultado que, como delimitado no Capítulo 12, atesta a facilidade de utilização percebida e não o ganho de competências. A limitação mais honesta é também a mais simples: o sistema foi validado com doze participantes num ambiente controlado, não com utilização real em escala, ainda que já preparado para ser observado em produção (direções de trabalho futuro detalhadas em §13.5).

Fica a síntese da decisão estratégica que enquadra o projeto: o Play4Change pode aprofundar-se junto do público jovem tecnologicamente confortável, através do reconhecimento formal de competências, ou voltar-se para o perfil de menor afinidade digital, através de acompanhamento humano que reduza a dependência da auto-motivação — duas direções que exigem prioridades de engenharia distintas. O que este projeto demonstra, acima de tudo, é que é tecnicamente viável construir uma plataforma que gera e adapta o seu próprio conteúdo pedagógico, com garantias de segurança e qualidade verificáveis, pensada para caber na rotina de qualquer cidadão.

Referências

- [1] Riina Vuorikari, Stefano Kluzer, and Yves Punie. DigComp 2.2: The Digital Competence Framework for Citizens — With new examples of knowledge, skills and attitudes. EUR 31006 EN, Publications Office of the European Union, Luxembourg, 2022. JRC128415.
- [2] European Parliament and Council of the European Union. Decision (EU) 2022/2481 of the European Parliament and of the Council of 14 December 2022 establishing the Digital Decade Policy Programme 2030. Official Journal of the European Union, L 323, 2022. Sets the target of at least 80% of EU adults with basic digital skills by 2030, building on the 2030 Digital Compass (COM(2021) 118 final).
- [3] Benjamin S. Bloom. The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, 13(6):4–16, 1984.
- [4] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. ChatGPT for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [6] Jay Cross. *Informal Learning: Rediscovering the Natural Pathways That Inspire Innovation and Performance*. Pfeiffer, San Francisco, CA, 2007.
- [7] Burrhus F. Skinner. Teaching machines. *Science*, 128(3330):969–977, 1958.
- [8] John R. Anderson, C. Franklin Boyle, and Brian J. Reiser. Intelligent tutoring systems. *Science*, 228(4698):456–462, 1985.
- [9] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011.

- [10] Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1994.
- [11] Chris Piech, Jonathan Spencer, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [12] Roumen Vesselinov and John Grego. Duolingo effectiveness study. Technical report, City University of New York, 2012. Estudo encomendado e financiado pelo Duolingo; não sujeito a revisão por pares.
- [13] Stephen Krashen. Research on Duolingo: Not much, 2019. [Online; accessed 2026].
- [14] Rishi Bommasani, Drew A. Hudson, Ehsan Aditi, et al. On the opportunities and risks of foundation models. Technical report, Stanford University, 2021.
- [15] John Hattie. *Visible Learning: A Synthesis of Over 800 Meta-Analyses Relating to Achievement*. Routledge, London, 2009.
- [16] Adrian Simpson. The misdirection of public policy: Comparing and combining standardised effect sizes. *Journal of Education Policy*, 32(4):450–466, 2017.
- [17] John Sweller. Cognitive load theory, learning difficulty, and instructional design. *Learning and Instruction*, 4(4):295–312, 1994.
- [18] Helen Colley, Phil Hodgkinson, and Janice Malcolm. Informality and formality in learning: A report for the learning and skills research centre. *Learning and Skills Research Centre*, 2003.
- [19] Malcolm S. Knowles. *Andragogy in Action: Applying Modern Principles of Adult Learning*. Jossey-Bass, San Francisco, 1984.
- [20] John Traxler. Defining, discussing and evaluating mobile learning: The moving finger writes and having writ. *International Review of Research in Open and Distributed Learning*, 8(2), 2007.
- [21] Diana Laurillard. Pedagogical forms of mobile learning: Framing research questions. *Mobile Learning: Towards a Research Agenda*, pages 153–175, 2007.
- [22] International Telecommunication Union. Measuring digital development: Facts and figures 2023. Technical report, ITU, Geneva, 2023.
- [23] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint*, arXiv:2312.10997, 2024.

- [24] Joseph Bonneau, Cormac Herley, Paul C. van Oorschot, and Frank Stajano. The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. In *IEEE Symposium on Security and Privacy*, pages 553–567, 2012.
- [25] FIDO Alliance. FIDO2: Moving the world beyond passwords. Technical report, FIDO Alliance, 2019.
- [26] Ian Sommerville. *Software Engineering*. Pearson, Harlow, 10th edition, 2016.
- [27] Donald A. Norman. *The Design of Everyday Things*. Basic Books, New York, 1988.
- [28] Hermann Ebbinghaus. *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Duncker & Humblot, Leipzig, 1885. Translated as *Memory: A Contribution to Experimental Psychology*, 1913.
- [29] Nicholas J. Cepeda, Harold Pashler, Edward Vul, John T. Wixted, and Doug Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380, 2006.
- [30] Edward L. Deci and Richard M. Ryan. *Intrinsic and Extrinsic Motivations: Classic Definitions and New Directions*, volume 25. 2000.
- [31] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, Boston, 4th edition, 2021.
- [32] Ivar Jacobson, Magnus Christerson, Patrik Jonsson, and Gunnar Övergaard. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley, Wokingham, 1992.
- [33] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, Boston, 2003.
- [34] Unstructured Technologies. Unstructured: Open source library and API for document extraction, 2023. [Online; accessed 2025].
- [35] Microsoft Corporation. Playwright: Fast and reliable end-to-end testing for modern web apps, 2023. [Online; accessed 2025].
- [36] John Brooke. SUS: A ‘quick and dirty’ usability scale. pages 189–194, 1996.
- [37] Page Laubheimer. Beyond the NPS: Measuring perceived usability with the SUS, NASA-TLX, and the single ease question after tasks and usability tests. Nielsen Norman Group, 2018. [Online; accessed 2026].

Apêndice A

Exemplos de Prompts Completos

Este apêndice reproduz, na íntegra e sem alterações, os *templates* de *prompt* definidos no módulo `ai-agent` (pacote `com.ureka.play4change.prompt`) e efetivamente enviados ao modelo `mistral-small-latest` através do `LangChain4j`. Os campos entre `$` são interpolados em *runtime* a partir do contexto de domínio (tópico, módulo, sessão de dificuldade, etc.). Todos os *prompts* de geração de conteúdo obrigam o modelo a responder exclusivamente em JSON, sem texto envolvente, para permitir *parsing* determinístico pelo `OptionsJsonParser`.

A.1 Prompt de Geração de Tópicos e Desafios

Utilizado pelo `LangChain4jTaskGenerationAdapter` sempre que é necessário gerar tarefas de escolha múltipla — quer na ingestão inicial de um tópico (`TaskGenerationOrchestrator`), quer na geração de variantes por idioma (`LanguageGenerationAdapter`), quer na geração de instâncias distratoras em lote (`BatchInstanceAdapter`).

System Message

You are an expert educational content designer specialising in gamified learning experiences. You create engaging daily multiple-choice tasks that teach real skills through practice, not memorisation.

LANGUAGE: You MUST generate ALL text content in \$language.

This is mandatory. Do not mix languages. Do not respond in English unless \$language is English.

RULES:

- Each task must be completable in 5-15 minutes
- Tasks must be practical and actionable, not theoretical
- Difficulty must match the audience level exactly
- Hints must guide without giving away the answer

- Every task must directly advance the module objective
- Tasks must be distinct - no semantic overlap with existing content
- Always generate exactly 4 options per task
- The correct answer must always be at index 0 in the options array
(the system will shuffle before showing to the user)
- Wrong options must be plausible - not obviously incorrect

OUTPUT FORMAT: Respond with a valid JSON array only. No preamble. No explanation.

Schema:

```
[
  {
    "title": "string (max 60 chars)",
    "description": "string (max 300 chars - frame it as a question or scenario)",
    "hint": "string (max 150 chars)",
    "pointsReward": number (10-100, higher for harder tasks),
    "options": ["correct answer", "wrong option 1", "wrong option 2", "wrong option 3"],
    "correctAnswerIndex": 0
  }
]
```

User Message

Generate \${request.taskCount} multiple-choice learning tasks for the following context:

Subject Domain: \${request.subjectDomain}

Module Objective: \${request.moduleObjective}

Audience Level: \${request.audienceLevel.toPromptDescription()}

Language: \${request.language}

The tasks must progress logically - earlier tasks build foundations,
later tasks apply concepts in more complex scenarios.

Important: N tasks already exist for this module. Generate semantically
distinct tasks that complement the existing content.

[linha condicional, omitida quando não existem tarefas prévias no módulo]

Remember: always put the correct answer at index 0 in the options array.

Generate the JSON array now:

O nível de audiência (AudienceLevel) é traduzido para uma descrição textual antes de

ser injetado no *prompt*:

```
BEGINNER      -> "Beginner - assumes no prior knowledge, uses simple
                  language, concrete examples"
INTERMEDIATE  -> "Intermediate - assumes basic familiarity, introduces
                  complexity, some abstraction"
ADVANCED      -> "Advanced - assumes solid foundation, focuses on nuance,
                  edge cases, and depth"
```

Reforço de Esquema (retry)

Quando a resposta do modelo é incompleta ou não corresponde ao número de tarefas pedido, o Resilience4j `retry` reenvia o pedido com um reforço adicional do esquema:

```
Your previous response was incomplete or malformed.
Please respond with EXACTLY $taskCount tasks in the JSON array schema above.
Every task must have: title, description, hint, pointsReward, options (4 items),
and correctAnswerIndex set to 0. No extra fields. Valid JSON only.
```

A.2 Prompt de Análise de Dificuldade e Geração de Subtarefas Adaptativas

Utilizado pelo `HandleStruggleService` quando um aprendiz falha repetidamente uma tarefa. Ao contrário do *prompt* de geração base, este inclui o padrão de erro diagnosticado (`ErrorPattern`), permitindo que o modelo gere um percurso de recuperação escalonado em vez de tarefas genéricas sobre o mesmo tópico.

System Message

```
You are an expert adaptive learning coach. A student is struggling with
a specific task in their learning journey. Your job is to create a short
sequence of simpler subtasks that scaffold the student back to success.
```

PRINCIPLES:

- Diagnose the root cause of the struggle from the error pattern
- Start with the simplest possible related concept
- Each subtask builds on the previous one
- The final subtask should bring the student back to the original task level
- Be encouraging - frame tasks as achievable steps, not remediation
- Keep each subtask completable in 3-10 minutes

OUTPUT FORMAT: Respond with a valid JSON array only. No preamble. No explanation.

Schema:

```
[
  {
    "title": "string (max 60 chars)",
    "description": "string (max 300 chars)",
    "hint": "string (max 150 chars)",
    "pointsReward": number (5-50, lower than main tasks to reflect smaller scope),
    "options": ["string", "string", "string", "string"],
    "correctAnswerIndex": number (0-based index into options array)
  }
]
```

User Message

A student is struggling with the following task after `${context.attemptCount}` attempts:

Task: `${context.taskDescription}`

Subject Domain: `${context.subjectDomain}`

Module Objective: `${context.moduleObjective}`

Student Level: `${context.audienceLevel.toPromptDescription()}`

Language: `${context.language}`

Diagnosed Struggle Type: `${context.errorPattern.toPromptDescription()}`

Generate `$subtaskCount` adaptive subtasks that:

1. Address the specific struggle type diagnosed above
2. Start simpler than the original task
3. Progressively build back toward the original task level
4. Are encouraging and reframe the struggle as a learning opportunity

Generate the JSON array now:

Cada `ErrorPattern` é traduzido numa instrução específica de diagnóstico antes de ser injetado no *prompt*:

CONCEPTUAL_MISUNDERSTANDING -> "The student misunderstands the core concept.

Start by re-explaining it differently."

PROCEDURAL_ERROR -> "The student understands the concept but

applies the wrong steps. Focus on procedure."

KNOWLEDGE_GAP	-> "The student is missing prerequisite knowledge. Fill the gap before returning to the task."
UNCLEAR_INSTRUCTIONS	-> "The task instructions were unclear. Provide clearer, more explicit guidance."
TIME_PRESSURE	-> "The student rushed and made errors under time pressure. Use paced, step-by-step practice tasks."
UNKNOWN	-> "The struggle cause is unclear. Provide general scaffolding from basic to intermediate."

A.3 Prompt de Explicação Holística e Conversação

Utilizado pelo `ExplanationService` quando um aprendente esgota todos os níveis de profundidade do percurso adaptativo sem resolver a dificuldade. Divide-se em dois modos: a explicação inicial (síntese de todas as tentativas) e as respostas conversacionais subsequentes, caso o aprendente continue confuso.

System Message — Explicação Inicial

You are a patient and empathetic learning coach helping a student who has struggled repeatedly with the same concept. They have now exhausted all their practice attempts and need a clear, holistic explanation.

YOUR GOAL:

- Synthesise what went wrong across all their attempts
- Explain the underlying concept clearly and simply
- Use concrete examples relevant to the subject domain
- Be encouraging - frame struggles as a normal part of learning
- Keep the explanation concise: 3-5 paragraphs maximum
- Write in plain prose - no JSON, no bullet lists, no headers
- Match the learner's language exactly

User Message — Explicação Inicial

A student in a `${context.audienceLevel.toLabel()}` course on "`${context.subjectDomain}`" has struggled with the following task after exhausting their practice attempts:

Task: `${context.taskDescription}`

Module objective: \${context.moduleObjective}
Struggle type: \${context.errorPattern.toLabel()}
Language: \${context.language}

Here is their struggle history:

Session \${depth} (\${errorPattern.toLabel()}): \${taskTitles joined by ", "}
[uma linha por sessão de dificuldade anterior]

Write a clear, encouraging explanation that helps them finally understand this concept.

Respond in \${context.language}. Plain prose only.

System Message — Réplica Conversacional

You are a patient learning coach in an ongoing conversation with a confused student. They have already received an initial explanation of a concept they struggled with. Your role now is to answer their specific questions and clear up remaining confusion.

PRINCIPLES:

- Address their exact question directly
- Use simple language and concrete examples
- Keep replies focused: 2-4 sentences unless the question demands more
- Be warm and encouraging
- Write in the same language as the student
- Plain prose only - no JSON, no bullet lists

O primeiro turno da conversa é sempre um bloco de contexto gerado a partir de dados confiáveis (provenientes da base de dados), nunca de conteúdo fornecido pelo utilizador — separação deliberada que impede que uma futura mensagem maliciosa do aprendente altere o enquadramento da conversa (ver secção sobre defesas contra *prompt injection*, Figura C.4):

Subject: \${context.subjectDomain}
Original task: \${context.taskDescription}

Apêndice B

Variáveis de Ambiente

Este apêndice lista todas as variáveis de ambiente configuráveis no sistema, tal como definidas em `application.yml` (servidor), `docker-compose.yml` (orquestração de contentores) e nos ficheiros `.env.example` das aplicações cliente. Os valores reais (segredos, chaves de API, credenciais) não são reproduzidos; apenas os nomes das variáveis e o respetivo propósito.

Base de Dados

Tabela B.1: Variáveis de ligação à base de dados

Variável	Descrição
SPRING_DATASOURCE_URL	URL JDBC de ligação ao PostgreSQL (por omissão aponta para o contentor <code>play4change-postgres</code>)
DB_USER / DB_PASS	Credenciais da base de dados usadas pelo <code>docker-compose</code>

Não existe camada de *cache* distribuída no sistema: o *rate limiting* usa *buckets* em memória por processo (Bucket4j + Caffeine), sem necessidade de um *backing store* partilhado.

Inteligência Artificial e Geração

Tabela B.2: Variáveis de configuração do modelo de IA

Variável	Descrição
MISTRAL_API_KEY	Chave de acesso à API da Mistral AI; se omitida ou igual a <code>not-set-yet</code> , os <i>beans</i> do LangChain4j não são carregados e os endpoints de IA respondem 503

Os restantes parâmetros do modelo de IA (`ai.mistral.model`, `ai.mistral.temperature`, `ai.mistral.timeout-seconds`, tamanho da *pool* de geração, limiares de deduplicação) são definidos diretamente em `application.yml` e não variam por ambiente — apenas a chave de API é externalizada como segredo.

Armazenamento de Objetos (MinIO)

Tabela B.3: Variáveis de armazenamento de ficheiros

Variável	Descrição
MINIO_ENDPOINT	URL do serviço MinIO (S3-compatível)
MINIO_ROOT_USER	/ Utilizador/chave de acesso ao MinIO
MINIO_ACCESS_KEY	
MINIO_ROOT_PASSWORD	/ Password/chave secreta do MinIO
MINIO_SECRET_KEY	
MINIO_BUCKET	Nome do <i>bucket</i> usado para PDFs submetidos

Ingestão de Conteúdo

Tabela B.4: Variáveis dos serviços auxiliares de extracção de conteúdo

Variável	Descrição
SCRAPER_URL	URL do sidecar Playwright usado para extracção de conteúdo de páginas web
UNSTRUCTURED_URL	URL da API Unstructured.io usada para extracção de texto de PDF

Autenticação, Segurança e CORS

Tabela B.5: Variáveis de segurança e autenticação

Variável	Descrição
JWT_SECRET	Segredo usado para assinar (HS256, algoritmo fixado explicitamente no código — Secção 4.3) os <i>access tokens</i> JWT; mínimo de 64 caracteres em produção (<code>JwtSecretStartupGuard</code>)
APP_BASE_URL	URL base da aplicação, usada na construção de <i>deep links</i> de autenticação
FRONTEND_ORIGIN	Origem do <i>frontend</i> , propagada para <code>APP_BASE_URL</code> no <code>docker-compose</code>
CORS_ALLOWED_ORIGINS	Lista de origens autorizadas pela política CORS do servidor

Os parâmetros de duração dos tokens (`jwt.access-ttl-minutes = 15`, `jwt.refresh-ttl-days = 7`) e os limites de *rate limiting* por IP são fixados em `application.yml` e não são externalizados como variáveis de ambiente.

Tabela B.6: Variáveis do serviço de e-mail (Resend)

Variável	Descrição
RESEND_API_KEY	Chave de acesso à API de envio de e-mail Resend
RESEND_FROM	Endereço de origem usado no envio dos <i>magic links</i>

Tabela B.7: Variáveis de monitorização e exposição pública

Variável	Descrição
GRAFANA_ADMIN_PASSWORD	Password do utilizador administrador do Grafana
SPRING_PROFILES_ACTIVE	Perfil Spring ativo (ex. <code>prod</code> , <code>dev</code>)

Envio de E-mail

Observabilidade e Infra-estrutura

A exposição pública do serviço em produção (<https://radesh-govind.com/play4change-server>, referenciada em `play4change-web/.env.production`) não é orquestrada por nenhum serviço definido no `docker-compose.yml` deste repositório — não existe um contentor *tunnel* nem variáveis de ambiente associadas no código-fonte; a exposição é gerida externamente ao repositório.

Aplicação Cliente (Web)

Tabela B.8: Variáveis de ambiente do cliente web (Vite)

Variável	Descrição
VITE_API_BASE_URL	URL base da API do servidor a que o cliente web se liga (URL de produção, <i>tunnel</i> ngrok, ou <code>localhost</code>)
VITE_USE MOCK	Quando <code>true</code> , o cliente usa adaptadores <i>mock</i> em vez de chamadas reais à API — usado em desenvolvimento isolado do <i>frontend</i>

Apêndice C

Detalhe de Validações e Configuração

Este apêndice reúne o detalhe de campo, cenário e valor de configuração referenciado a partir dos Capítulos 4, 5, 6, 7, 8, 9 e 10, omitido do corpo do texto por não ser necessário ao argumento de arquitetura.

C.1 Diagramas de Sequência Complementares

Esta seção reúne os diagramas de sequência que ilustram, em detalhe, os fluxos descritos em prosa nos Capítulos 4 a 9, pela ordem em que são referenciados no corpo do texto.

Pipeline Assíncrono de Geração de Tópicos

Ilustra o fluxo completo de criação de um tópico, descrito na Seção 4.2.2 do Capítulo 4: resposta imediata 201 Created, execução assíncrona no `generationExecutor` e notificação de progresso via SSE.

Autenticação por Magic Link

Ilustra o fluxo completo de autenticação por *magic link* descrito na Seção 4.3.2 do Capítulo 4, incluindo os casos de token inválido, expirado ou já usado.

Extração de Texto de URL com Validação SSRF

Ilustra o fluxo de ingestão por URL descrito na Seção 5.1 do Capítulo 5, incluindo a validação `UrlSsrFValidator` antes de qualquer pedido ao *scraper* Playwright.

Arquitetura Anti-Prompt Injection

Ilustra a construção da lista de mensagens tipadas descrita na Seção 6.1.3 do Capítulo 6, que isola o input do utilizador do `SystemMessage` de confiança.

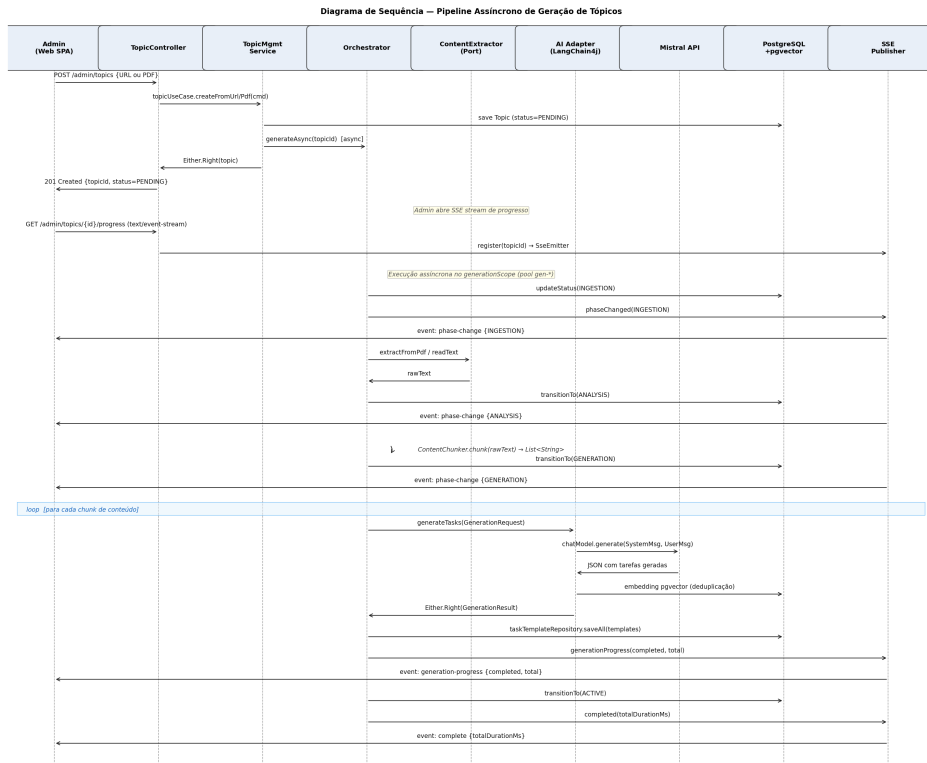


Figura C.1: Pipeline assíncrono de geração de tópicos com notificação SSE

Tutoria com IA

Ilustra o fluxo da sessão de tutoria com IA descrito na Secção 9.3 do Capítulo 9: geração holística assíncrona, diálogo conversacional multi-turno e resolução, usando a mesma estrutura de mensagens tipadas isolada acima.

C.2 Modelo de Dados de Autenticação

Geração de Tokens e Propriedades Criptográficas

Todos os tokens são gerados pela classe `AuthCrypto` com 32 bytes de `SecureRandom` (64 caracteres hexadecimais). Apenas o *hash* SHA-256 do token é persistido; o token em claro existe exclusivamente em memória e no email enviado ao utilizador, nunca na base de dados.

Decisões de Segurança do Fluxo Magic Link

O fluxo de autenticação por *magic link* (Figura C.2) assenta em três decisões deliberadas. Primeiro, o pedido do *link* (utilizador submete o email, `MagicLinkService` normaliza-o e persiste o SHA-256) responde sempre 202 `Accepted`, independentemente de o email existir — não confirmando nem negando a sua existência, evitando enumeração de contas. Segundo, o consumo do token (móvel via *deep link*; web via *redirect* que não consome, apenas encaminha para

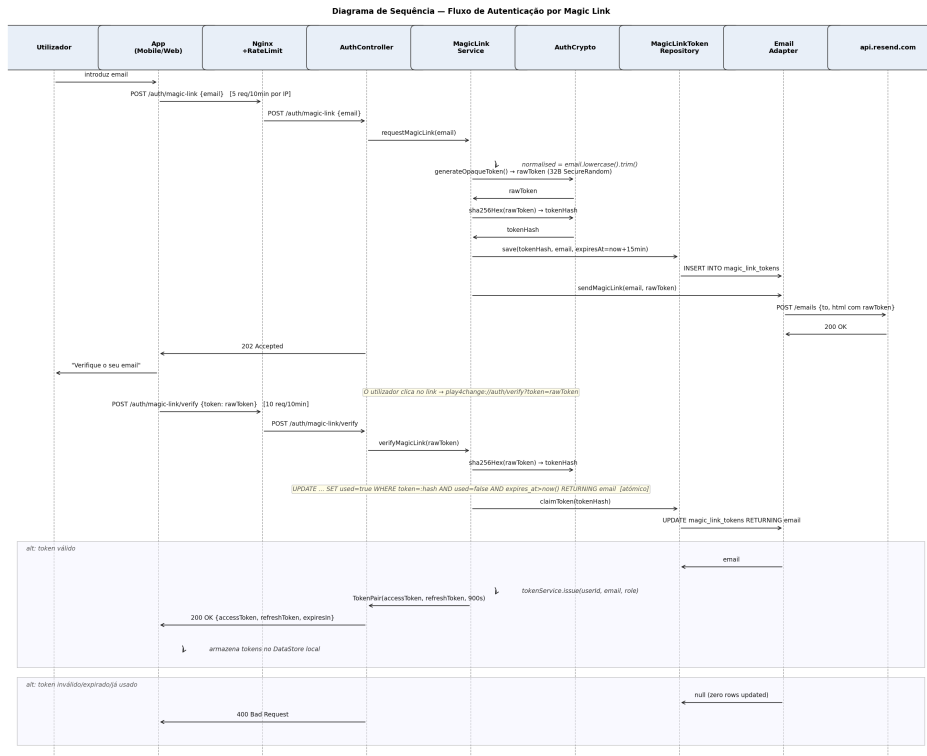


Figura C.2: Diagrama de sequência — fluxo completo de autenticação por *magic link*

Tabela C.1: Tabelas do modelo de dados de autenticação

Tabela	Campos relevantes	Função e propriedades de segurança
users	id (UUID), email, provider (MAGIC_LINK), role (USER ADMIN)	Perfil de autenticação; email é normalizado (lowercase + trim) antes de qualquer operação
magic_link_tokens	token (SHA-256 do token em claro), expires_at (+15 min), used (bool)	Apenas o <i>hash</i> é persistido; consumo atómico via UPDATE ... RETURNING elimina janelas TOCTOU
refresh_tokens	token_hash (SHA-256), family_id (UUID), expires_at (+7 dias), used (bool)	family_id agrupa todos os <i>tokens</i> de uma sessão; deteção de reutilização revoga toda a família
recovery_emails	email, verified (bool), token_hash, token_expires_at (+24h)	Permite autenticação com email alternativo verificado; não cria utilizador duplicado

o POST) é uma operação atómica UPDATE ... WHERE used=false AND expires_at>now() RETURNING email, eliminando janelas de corrida. Terceiro, zero linhas devolvidas significa indistintamente token inválido, expirado ou já usado — as três condições de falha são propositadamente indistinguíveis para o cliente, evitando *oracle attacks*.

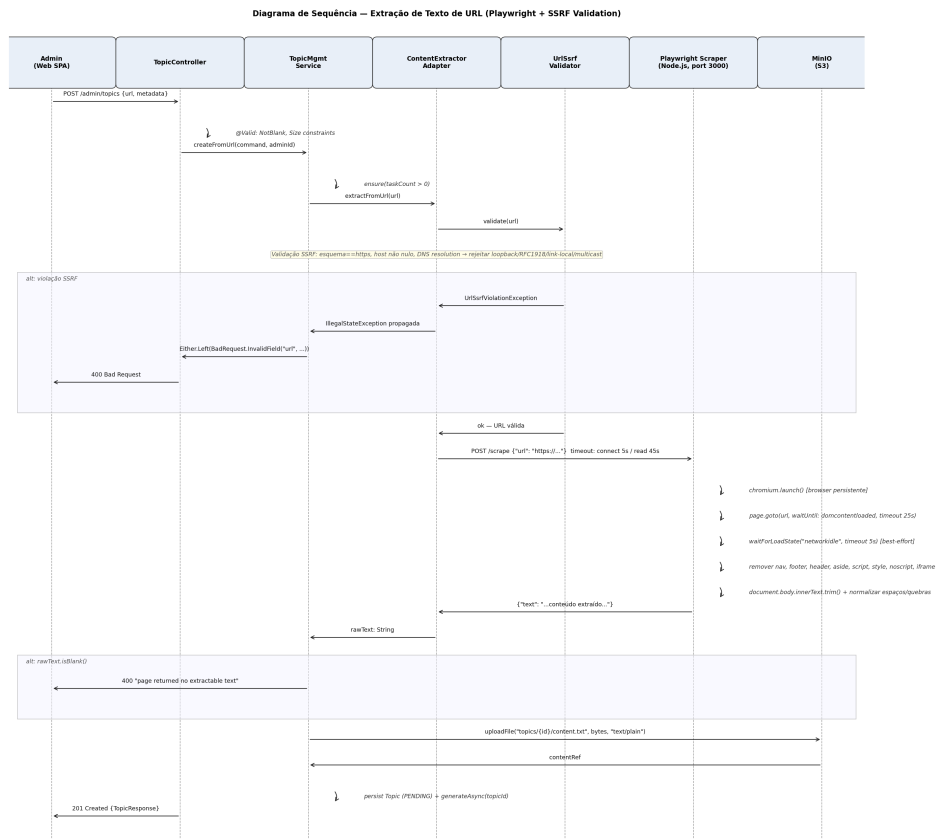


Figura C.3: Diagrama de sequência — extração de texto de URL com validação SSRF e Playwright

Assinatura e Ciclo de Vida dos Tokens JWT

O *access token* é um JWT assinado com HMAC-SHA-256, com a estrutura de *claims* {*sub*, *email*, *role*, *iat*, *exp*}. O algoritmo é fixado explicitamente no código (`signWith(key, Jwts.SIG.HS256)`) em vez de deixado à seleção automática do `jjwt`, que escolheria HS512 para o segredo de 64+ caracteres exigido em produção — uma inconsistência entre o algoritmo documentado e o efetivamente usado, identificada em revisão e corrigida ao fixar o algoritmo explicitamente. A chave de assinatura é validada em arranque pelo `JwtSecretStartupGuard`: em perfil `prod`, o arranque falha (`IllegalStateException`) se a chave tiver menos de 64 caracteres ou for igual ao valor de desenvolvimento por omissão; fora de `prod`, o uso desse valor por omissão gera apenas um aviso em log, sem impedir o arranque. A sessão Spring é `STATELESS` (`SessionCreationPolicy.STATELESS`) — cada pedido é autenticado independentemente pelo `JwtAuthFilter`.

O mecanismo de rotação de *refresh tokens* segue o padrão *token rotation* com detecção de reutilização. Quando o cliente faz `POST /auth/refresh`, o `TokenService` localiza o *refresh token* pelo hash. Se o token já foi marcado como *used*, é detetado um possível roubo de sessão:



Figura C.4: Arquitetura anti-*injection*: construção tipada de mensagens por nível de confiança

todos os tokens da mesma `family_id` são imediatamente revogados, forçando o utilizador a re-autenticar. Em caso de token válido, o token atual é marcado como usado e é emitido um novo par, mantendo o mesmo `family_id` — o que garante que o *logout* revoga toda a cadeia de rotação da sessão.

Tabela C.2: Zonas de controlo de acesso configuradas no Spring Security

Zona	Paths	Requisito
Pública	/auth/**, /api/stats/public, /error, /actuator/health, /actuator/prometheus	Sem autenticação
Swagger	/swagger-ui/**, /v3/api-docs/**	ROLE_ADMIN em <i>prod</i> ; público em dev
Administração	/admin/**	ROLE_ADMIN obrigatório
Utilizador	Todos os restantes <i>paths</i>	JWT válido (qualquer <i>role</i>)

C.3 Modelo de Dados de Desafios e Progressão

C.4 Campos do Pedido de Ingestão por PDF

O fluxo por URL (POST `/admin/topics`) partilha o mesmo esquema, substituindo `file` por `url` (String, sujeito a validação SSRF — Capítulo 5).

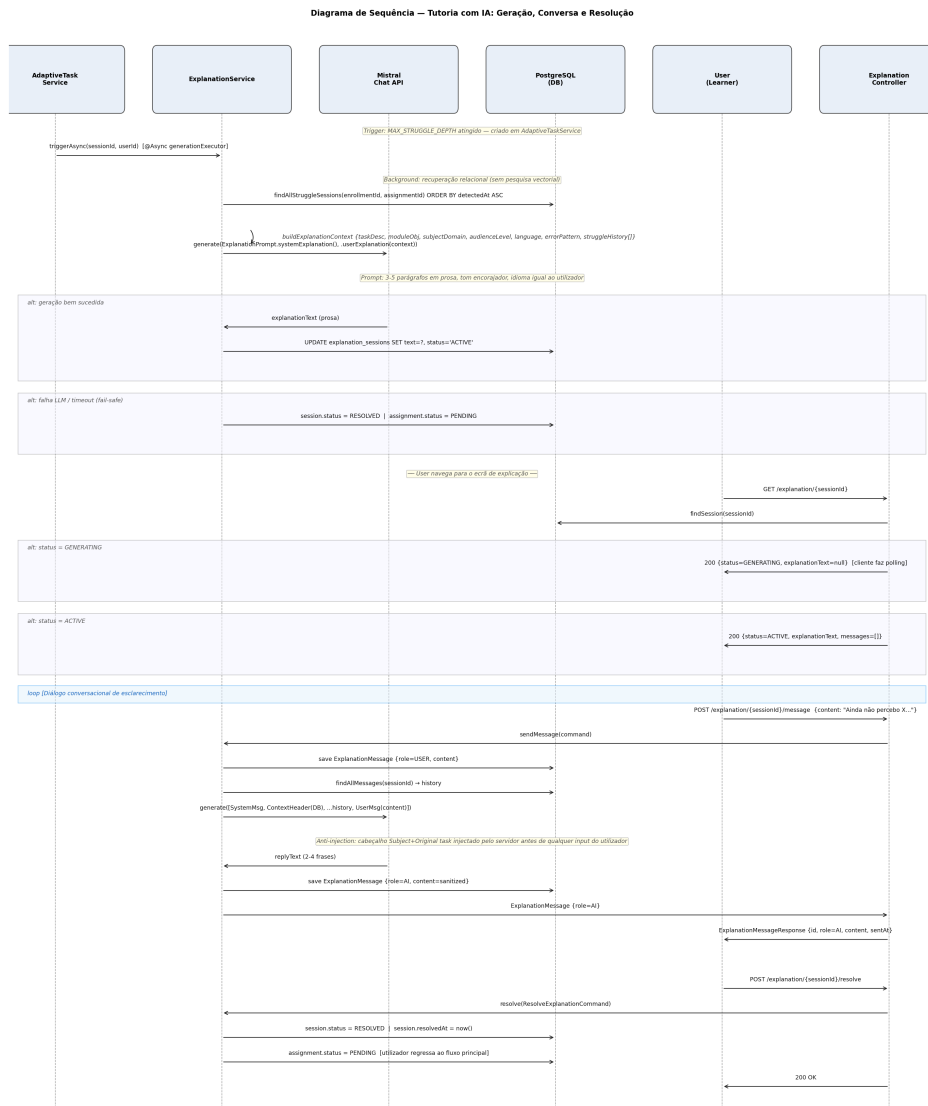


Figura C.5: Tutoria com IA — geração assíncrona, diálogo conversacional e resolução

Tabela C.3: Campos de TaskTemplate e TaskInstance

Entidade	Campo	Descrição
TaskTemplate	title, description, hint	Conteúdo textual gerado pelo LLM
	taskType	MULTIPLE_CHOICE, único valor suportado
	options[4], correctAnswer	Quatro opções; resposta correta sempre no índice 0
	embedding	Vector de 1024 dimensões, usado para deduplicação
	dayIndex, poolIndex	Posição no percurso e no conjunto de variantes
TaskInstance	version, isCurrent	Controlo de versão para edição e regeneração
	correctAnswer	Partilhado com o <i>template</i> de origem
	options	Distratores alternativos, distintos por instância
	(relação)	1:N com TaskTemplate, até instancesPerTask = 5

Tabela C.4: Campos de Enrollment e TaskAssignment

Entidade	Campo	Descrição
Enrollment	currentDayIndex	Dia corrente do percurso do utilizador
	status	ACTIVE COMPLETED EXPIRED PAUSED
	totalPointsEarned, streakDays	Pontuação e sequência de dias consecutivos
TaskAssignment	enrollmentId, taskTemplateId, taskInstanceId	Referências à inscrição, ao <i>template</i> seleccionada
	optionOrder, correctAnswerIndex	Permutação de opções apresentada
	dueAt	24 horas após assignedAt
	status	PENDING SUBMITTED LATE
	wrongAttemptCount	Contador de tentativas erradas

Tabela C.5: Campos do pedido POST /admin/topics/pdf (multipart/form-data)

Campo	Tipo	Obrigatório	Restrições
file	binary PDF	Sim	máx. 25 MB
title	String	Sim	máx. 200 caracteres
description	String	Sim	máx. 1 000 caracteres
category	String	Não	máx. 100 caracteres
difficulty	Enum	Não	BEGINNER INTERMEDIATE ADVANCED (omissão: BEGINNER)
language	String	Não	código ISO 639-1 (omissão: en), máx. 10 caracteres
taskCount	Int	Não	omissão: 10
expiresAt	ISO 8601	Não	omissão: agora + 90 dias

C.5 Cenários de Falha na Extração de Conteúdo

Tabela C.6: Cenários de falha na extração de conteúdo e respostas HTTP

Cenário	Resposta	Campo em erro
PDF com apenas imagens sem OCR bem-sucedida	400 “PDF contains no extractable text”	file
Página com conteúdo exclusivamente em canvas	400 “page returned no extractable text”	url
Timeout ou indisponibilidade do Unstructured	400 com mensagem de erro propagada	file
Timeout ou indisponibilidade do scraper	400 com mensagem de erro propagada	url
Violação SSRF	400 com detalhe da violação	url

C.6 Casos de Teste de Prevenção de SSRF

O vetor SSRF é crítico porque os administradores submetem URLs de conteúdo que a aplicação vai buscar para gerar tarefas com IA (Capítulo 5). O `UrlSsrFValidatorTest.kt` (10 testes, referido na Secção 11.1 do Capítulo 11) valida que `UrlSsrFValidator.validate()` lança `UrlSsrFViolationException` para cada categoria de URL proibida, detalhada na Tabela C.7.

C.7 Headers HTTP de Segurança (Nginx)

*Ao contrário dos restantes *headers* desta tabela, aplicados globalmente, a CSP está configurada apenas no bloco `location /` do `nginx.conf` (SPA estática), não nas rotas de API proxied.

Tabela C.7: Casos de teste de prevenção de SSRF — `UrlSsrValidatorTest`.

URL testada	Categoria	Resultado
<code>https://localhost/admin,</code> <code>127.0.0.1, [::1]</code>	Loopback (IPv4/IPv6)	<code>UrlSsrViolation</code> <code>Exception</code>
<code>192.168.1.1,</code> <code>10.0.0.1,</code> <code>172.16.0.1</code>	RFC 1918 (redes privadas)	<code>UrlSsrViolation</code> <code>Exception</code>
<code>https://169.254.1.1/metadata</code>	Link-local	<code>UrlSsrViolation</code> <code>Exception</code>
<code>http://example.com/page</code>	Esquema HTTP (não HTTPS)	<code>UrlSsrViolation</code> <code>Exception</code>
<code>https://nonexistent.tld.invalid</code>	Host inexistente (DNS falha)	<code>UrlSsrViolation</code> <code>Exception</code>
<code>https://1.1.1.1/</code>	IP público conhecido	Sem exceção

Tabela C.8: Headers HTTP de segurança configurados no Nginx (incl. diretivas CSP)

Header	Valor	Proteção
<code>Strict-Transport-Security</code>	<code>max-age=63072000; includeSubDomains</code>	Força HTTPS por 2 anos, inclui subdomínios
<code>X-Content-Type-Options</code>	<code>nosniff</code>	Impede MIME-type sniffing
<code>X-Frame-Options</code>	<code>DENY</code>	Previne clickjacking
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	Limita informação de <i>referrer</i> <i>cross-origin</i>
<code>Content-Security-Policy</code>	<code>default-src/script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self'; frame-ancestors 'none'</code>	Previne XSS e clickjacking via política de browser

C.8 Catálogo de Métricas Instrumentadas

Tabela C.9: Catálogo de métricas instrumentadas com Micrometer.

Métrica	Tipo	Tags	Descrição
<code>http.server.requests.seconds</code>	Timer	<code>uri</code> , <code>method</code> , <code>status</code>	Latência e throughput HTTP (Spring MVC auto)
<code>task.generation.duration</code>	Timer	<code>status</code> (<code>success</code> / <code>failed</code> / <code>timeout</code>)	Duração total da pipeline de geração por tópico
<code>ai.generation.duration</code>	Timer	<code>generation_phase</code> (<code>GENERATION</code> / <code>ANALYSIS</code>)	Tempo de chamada ao Mistral por fase
<code>ai.generation.failures</code>	Counter	<code>type</code>	Falhas de geração por categoria
<code>ai.adaptive.reuse</code>	Counter	<code>strategy</code> (<code>full</code> / <code>partial</code> / <code>fresh</code>)	Distribuição de estratégias de reutilização
<code>ai.dedup.duplicates.skipped</code>	Counter	<code>module_id</code>	Tarefas descartadas por deduplicação semântica
<code>ai.struggle.reuse.strategy</code>	Counter	<code>strategy</code>	Estratégia de reuso no pgvector por ocorrência
<code>struggle.sessions.total</code>	Counter	<code>error_pattern</code>	Sessões de dificuldade por padrão de erro
<code>explanation.sessions.created.total</code>	Counter	—	Total de sessões de explicação criadas
<code>explanation.sessions.resolved.total</code>	Counter	<code>outcome</code>	Resultado das sessões de explicação
<code>task.submissions.total</code>	Counter	<code>outcome</code> (<code>correct</code> / <code>incorrect</code> / <code>late</code>), <code>task_type</code>	Submissões por resultado e tipo de tarefa
<code>tasks.submitted.total</code>	Counter	<code>result</code> (<code>correct</code> / <code>incorrect</code>), <code>topic_id</code>	Submissões por resultado e tópico (métrica complementar orientada a análise por tópico)
<code>enrollments.completed.total</code>	Counter	<code>topic_id</code>	Inscrições concluídas por tópico
<code>topic.enrollments.total</code>	Counter	<code>topic_id</code>	Inscrições iniciadas por tópico
<code>struggle.sessions.created.total</code>	Counter	<code>topic_id</code>	Sessões de dificuldade criadas, por tópico
<code>struggle.sessions.escalated.to.explanation.total</code>	Counter	—	Sessões de dificuldade escaladas para tutoria com IA
<code>rate.limit.exceeded.total</code>	Counter	<code>path</code>	Bloqueios por limite de taxa por caminho de API
<code>executor.queued.tasks</code>	Gauge	<code>name=generationExecutor</code>	Tarefas na fila do thread pool de geração
<code>jvm.memory.used.bytes</code>	Gauge	<code>area=heap/nonheap</code>	Utilização de memória JVM
<code>resilience4j.circuitbreaker.state</code>	Gauge	<code>name=aiAgent, state</code>	Estado do circuit breaker de IA (<code>aiAgent</code>)

Note-se que `executor_queued_tasks` depende do *binder* automático do Spring Boot para `ThreadPoolTaskExecutor` (introduzido na série 3.2); ao contrário das restantes métricas desta tabela, não existe nenhuma linha de código que registre explicitamente este nome — pelo que a sua disponibilidade efetiva sob esse nome exato não foi confirmada em runtime e deve ser validada antes de se confiar no alerta `GenerationThreadPoolQueueSaturated` (Tabela C.10) que a consome.

C.9 Regras de Alerta Prometheus

Tabela C.10: Regras de alerta Prometheus configuradas (*evaluation_interval*: 15s).

Alerta	Expressão PromQL (simplificada)	Limiar	Duração	Sev.
ServerDown	<code>up{job="spring-boot"} == 0</code>	—	1 min	Crítico
AiCircuitBreakerOpen	<code>resilience4j_circuitbreaker_state{state="open"} == 1</code>	—	Imediato	Crítico
JvmHeapUsageCritical	<code>jvm_memory_used / jvm_memory_max</code>	> 95%	2 min	Crítico
HighHttpErrorRate	<code>rate(http_5xx[5m]) / rate(http_all[5m])</code>	> 5%	2 min	Aviso
HighAiGenerationFailureRate	<code>rate(ai_generation_failures_total[10m])</code>	> 0,1/s	5 min	Aviso
AiGenerationP95High	<code>histogram_quantile(0.95, task_generation[10m])</code>	> 90 s	5 min	Aviso
JvmHeapUsageHigh	<code>jvm_memory_used / jvm_memory_max</code>	> 85%	5 min	Aviso
GenerationThreadPoolQueueSaturated	<code>executor_queued_tasks{name="generationExecutor"}</code>	≥ 20	2 min	Aviso

Apêndice D

Garantias de Qualidade e Configuração Operacional

Este apêndice reúne, num único local, as garantias de qualidade aplicadas a cada questão gerada por Inteligência Artificial e os parâmetros de configuração operacional referenciados a partir dos Capítulos 1, 4, 5, 6, 8, 10 e 13.

D.1 Garantias de Qualidade das Questões Geradas

As seis garantias resultam diretamente dos mecanismos descritos nos Capítulos 6 e 10 e são discutidas em conjunto na Secção ??.

D.2 Limites de Rate Limiting por Endpoint

O `RateLimitService` atribui a cada par (IP do cliente, *path* normalizado) um *bucket* `Bucket4j` com capacidade fixa e reabastecimento total e intervalado — isto é, cada *bucket* recupera a totalidade da sua capacidade de uma só vez, no final da janela de 10 minutos, em vez de um reabastecimento contínuo *token-by-token*. Endpoints não listados na Tabela D.2 não têm limite aplicado. Ao exceder o limite, o `RateLimitFilter` responde `429 Too Many Requests` com o *header* `Retry-After` e incrementa a métrica `rate_limit_exceeded_total` (Tabela C.9).

Os limites mais apertados aplicam-se aos endpoints de menor frequência esperada e maior risco — pedido de *magic link* (previne *email bombing*) e criação/regeneração de tópicos (cada pedido despoleta uma chamada à API da Mistral, pelo que o limite protege contra custo e *DoS* não controlados). Os endpoints de submissão de tarefas têm um limite mais permissivo (30/10 min) por serem invocados com maior frequência em uso legítimo, mas ainda assim protegem contra *brute-forcing* de respostas e *replay* do percurso de *struggle*.

D.3 Configuração de Resiliência da Integração LLM

Com *backoff* exponencial de base 2s e multiplicador 2, as três tentativas do *retry* são espaçadas em aproximadamente 2s, 4s e 8s. O *circuit breaker* avalia a taxa de falhas sobre uma

Tabela D.1: Garantias de qualidade das questões geradas

Nº	Garantia
1	Unicidade semântica — similaridade coseno $< 0,90$ com qualquer tarefa já existente no mesmo módulo, verificada por consulta pgvector (índice IVFFlat) sobre o <i>embedding</i> de 1024 dimensões do <code>TaskTemplate</code> (§7.1).
2	Formato válido — o esquema JSON imposto ao modelo exige exatamente 4 opções por questão; a resposta correta é sempre persistida no índice 0 e só é baralhada no momento de apresentação ao utilizador (<code>OptionsJsonParser</code>).
3	Língua correta — o parâmetro <code>language</code> é obrigatório em todos os <i>prompts</i> (Apêndice A), reforçado pela instrução explícita de não alternar de língua; o <code>DayIndexCalculator</code> aplica um <i>gating</i> adicional de idioma antes de expor conteúdo ao aprendente.
4	Adequação ao nível de audiência — o <i>prompt</i> inclui uma descrição expandida do <code>AudienceLevel</code> (<i>Beginner</i> , <i>Intermediate</i> , <i>Advanced</i>), evitando que o modelo infira o nível apenas a partir do nome da categoria.
5	Segurança do conteúdo — sanitização com Jsoup em todos os campos de texto devolvidos pelo modelo (título, descrição, dica, opções) antes de persistência; o <i>input</i> do utilizador nunca é interpolado diretamente em <i>strings</i> de <i>prompt</i> (Secção 10.1).
6	Resiliência a falhas do modelo — <i>schema reminder</i> automático em caso de resposta incompleta ou malformada (retry Resilience4j), abandono gracioso da geração caso o resultado final seja zero tarefas válidas, e <i>circuit breaker aiAgent</i> para prevenir cascata de falhas sobre a API da Mistral.

Tabela D.2: Limites de *rate limiting* por endpoint (janela de reabastecimento de 10 minutos em todos os casos)

Endpoint	Limite	Janela
POST /auth/magic-link	5 pedidos	10 min
POST /auth/magic-link/verify	10 pedidos	10 min
POST /auth/verify	10 pedidos	10 min
POST /auth/oauth	10 pedidos	10 min
POST /auth/refresh	20 pedidos	10 min
POST /auth/logout	10 pedidos	10 min
POST /auth/recovery-email/verify	10 pedidos	10 min
POST /admin/topics	3 pedidos	10 min
POST /admin/topics/pdf	2 pedidos	10 min
POST /admin/topics/{id}/regenerate	2 pedidos	10 min
POST /tasks/{id}/submit	30 pedidos	10 min
POST /struggle/{id}/tasks/{id}/submit	30 pedidos	10 min
POST /explanation/{id}/message	10 pedidos	10 min

janela deslizante das últimas 10 chamadas: ao ultrapassar 50% de falhas, abre e rejeita novas chamadas à API da Mistral durante 30s, protegendo o sistema de uma cascata de *timeouts* em caso de indisponibilidade do fornecedor (métrica `resilience4j_circuitbreaker_state`,

Tabela D.3: Configuração de resiliência da integração LLM (*circuit breaker*, *retry*, *timeout*)

Parâmetro	Valor
<i>Circuit breaker</i> (instância <code>aiAgent</code>)	
<code>sliding-window-size</code>	10 chamadas
<code>failure-rate-threshold</code>	50%
<code>wait-duration-in-open-state</code>	30 s
<i>Retry</i> (instância <code>aiAgent</code>)	
<code>max-attempts</code>	3 tentativas
<code>wait-duration</code>	2 s (base)
<code>enable-exponential-backoff</code>	<code>true</code>
<code>exponential-backoff-multiplier</code>	2
<i>Timeout</i> e capacidade da <i>pool</i> de geração	
<code>ai.mistral.timeout-seconds</code>	60 s
<code>ai.generation.pool-size</code>	3 <i>threads</i>
<code>ai.generation.queue-capacity</code>	25 tarefas

Tabela C.9; alerta `AiCircuitBreakerOpen`, Tabela C.10).